

Министерство науки и высшего образования Российской Федерации
Санкт-Петербургский политехнический университет Петра Великого
Институт компьютерных наук и кибербезопасности
Высшая школа технологий искусственного интеллекта

ОТЧЁТ
о лабораторной работе № 1
«Генерация бинарного кода Грея.
Мультимножества и операции над ними»
по дисциплине «Дискретная математика»

Выполнил

студент гр. 5130201/50302

Р. Ю. Дужнов

Проверил

преподаватель

А. В. Востров

«___» _____ 2026 г.

Санкт-Петербург
2026

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	3
1 Математическое описание	5
1.1 Множества	5
1.2 Мультимножества	5
1.3 Операции над мультимножествами	6
1.4 Универсум на коде Грея	7
1.5 Алгоритм генерации бинарного кода Грея	8
1.6 Случайное заполнение мультимножества заданной мощности	10
2 Особенности реализации	12
2.1 Средства разработки и структура программы	12
2.2 Ограничения разрядности и вывода	13
2.3 Исключения	14
2.4 Модуль случайных чисел	15
2.5 Генерация кода Грея и носитель	16
2.6 Класс Multiset	18
2.7 Построение универсума	21
2.8 Модуль ввода	21
2.9 Класс Menu и функция main	22
3 Результаты работы	26
3.1 Сценарий 1: случайная кратность, автоматическое заполнение	26
3.2 Сценарий 2: ручной ввод с ошибками и отмена	30
3.3 Сценарий 3: граничные мощности A и B	32
3.4 Сценарий 4: некорректный ввод команд и параметров универсума	34
3.5 Сценарий 5: разрядность $n = 0$	36
3.6 Сценарий 6: пересоздание универсума и конец ввода	38
3.7 Сценарий 7: наибольшая разрядность $n = 19$	39
ЗАКЛЮЧЕНИЕ	41
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ	44
ПРИЛОЖЕНИЕ А Текст программы	45

ВВЕДЕНИЕ

Требуется реализовать программу, которая генерирует бинарный код Грея заданной пользователем разрядности n и заполняет им универсум мультимножеств. На основе универсума формируются два мультимножества A и B ; для каждого пользователь выбирает способ заполнения (вручную или автоматически) и задаёт мощность. Программа выводит результаты операций: объединение, пересечение, разность (оба варианта), симметрическая разность, дополнение (оба варианта), арифметические сумма, разность, произведение и деление. Требуются пользовательское меню и защита от некорректного ввода.

В работе приняты следующие уточнения постановки.

- а) Универсум U — мультимножество: каждый код Грея входит в него с одной и той же кратностью $k \geq 1$. Кратность k вводит пользователь или выбирает программа случайно.
- б) A и B — подмультимножества U : $k_A(x) \leq k_U(x)$ для всех x . Мощность $|A|$ вводится пользователем или выбирается случайно; в обоих случаях $0 \leq |A| \leq |U|$, поэтому допустимо любое подмультимножество U , включая $\emptyset$ и сам U .
- в) Некоммутативные арифметические операции (разность и деление) вычисляются в обоих вариантах, поэтому результатов тринадцать.
- г) При $n = 0$ универсум пуст: $U = \{\}$, и все операции дают $\{\}$ (обоснование — в подразделе 1.4).

Входные данные программы перечислены в таблице 1. Пустая строка (только клавиша Enter) там, где она допустима, означает либо случайный выбор, либо отмену; в остальных местах она считается ошибкой.

Таблица 1 — Входные данные программы

Величина	Допустимые значения	Пустая строка
Команда меню	одна из букв <i>u a b t o m h q</i> (регистр не важен)	ошибка
Разрядность кода Грея <i>n</i>	целое от 0 до 19	ошибка
Кратность элементов универсума <i>k</i>	целое от 1 до 100	случайная кратность
Мощность $ A $, $ B $	целое от 0 до $ U $	случайная мощность
Способ заполнения <i>A</i> , <i>B</i>	буква <i>r</i> (случайное) или <i>m</i> (ручное)	отмена заполнения
Строка ручного заполнения	пары код : кратность или код через пробел; код — <i>n</i> символов 0 и 1, кратность — целое ≥ 1	отмена заполнения

Результат считается достигнутым, если для любых допустимых входных данных программа выводит U , A , B и тринадцать результатов операций в точном соответствии с определениями, а на любой некорректный ввод отвечает сообщением и повторным запросом, не завершаясь аварийно.

1 Математическое описание

1.1 Множества

Множество — совокупность различных элементов [1]. Если все рассматриваемые множества являются подмножествами одного множества U , его называют универсумом. Операции над $A, B \subseteq U$ [1]:

$$A \cup B = \{x \mid x \in A \text{ или } x \in B\}, \quad (1)$$

$$A \cap B = \{x \mid x \in A \text{ и } x \in B\}, \quad (2)$$

$$A \setminus B = \{x \mid x \in A \text{ и } x \notin B\} = A \cap \bar{B}, \quad (3)$$

$$A \triangle B = (A \setminus B) \cup (B \setminus A), \quad (4)$$

$$\bar{A} = U \setminus A. \quad (5)$$

Дополнение (5) определено только относительно универсума. Тождества $A \setminus B = A \cap \bar{B}$ и (4) переносятся на мультимножества и используются в реализации.

1.2 Мультимножества

Мультимножество $\hat{X}$ над множеством $X = \{x_1, \dots, x_s\}$ — совокупность элементов X , в которую x_i входит $a_i \geq 0$ раз [1]. Множество X называют носителем, число a_i — кратностью (показателем) элемента x_i . Запись:

$$\hat{X} = \{x_1^{a_1}, \dots, x_s^{a_s}\}, \quad (6)$$

элементы с нулевой кратностью не записываются. Мультимножество A над X однозначно задаётся функцией кратности $k_A: X \rightarrow \{0, 1, 2, \dots\}$, а его мощность — сумма кратностей:

$$|A| = \sum_{x \in X} k_A(x). \quad (7)$$

Мультимножество, все кратности которого равны 0 или 1, совпадает с обычным множеством.

Универсум U в данной работе тоже мультимножество над носителем X , все кратности которого не меньше 1. Мультимножество A называется под-

мультимножеством U ($A \subseteq U$), если

$$k_A(x) \leq k_U(x) \quad \text{для всех } x \in X. \quad (8)$$

Мультимножества A и B рассматриваются только как подмультимножества U . Из (7) и (8) следует $0 \leq |A| \leq |U|$, причём каждое значение из этого отрезка достигается: подмультимножество мощности m получается, если взять любые m из $|U|$ «единиц кратности» универсума.

1.3 Операции над мультимножествами

Все операции выполняются поэлементно: кратность x в результате зависит только от $k_A(x)$, $k_B(x)$ и $k_U(x)$ [1, 2]. Функции кратности результатов приведены в таблице 2.

Таблица 2 — Операции над мультимножествами

Операция	Обозначение	Кратность x в результате
Объединение	$A \cup B$	$\max\{k_A(x), k_B(x)\}$
Пересечение	$A \cap B$	$\min\{k_A(x), k_B(x)\}$
Дополнение	$\bar{A}$	$k_U(x) - k_A(x)$
Разность	$A \setminus B$	$\min\{k_A(x), k_U(x) - k_B(x)\}$
Симметрическая разность	$A \triangle B$	$\max\{\min\{k_A(x), k_U(x) - k_B(x)\}, \min\{k_B(x), k_U(x) - k_A(x)\}\}$
Арифметическая сумма	$A + B$	$\min\{k_A(x) + k_B(x), k_U(x)\}$
Арифметическая разность	$A - B$	$\max\{k_A(x) - k_B(x), 0\}$
Арифметическое произведение	$A * B$	$\min\{k_A(x) \cdot k_B(x), k_U(x)\}$
Арифметическое деление	A/B	$\lfloor k_A(x)/k_B(x) \rfloor$ при $k_B(x) > 0$; 0 при $k_B(x) = 0$

Формула разности получена из $A \setminus B = A \cap \bar{B}$, симметрической разности — из (4). Арифметическая разность не зависит от универсума и отличается от теоретико-множественной: например, при $k_U(x) = 5$, $k_A(x) = 3$, $k_B(x) = 1$ имеем $k_{A \setminus B}(x) = \min\{3, 4\} = 3$, но $k_{A-B}(x) = 2$. Сумма и произведение ограничены сверху кратностью $k_U(x)$, иначе результат мог бы не быть подмультимножеством U и к нему нельзя было бы применить дополнение.

Утверждение. Если $A, B \subseteq U$, то результат каждой операции из таблицы 2 является подмультимножеством U .

Действительно: $\max\{k_A(x), k_B(x)\} \leq k_U(x)$ и $\min\{k_A(x), k_B(x)\} \leq k_U(x)$ в силу (8); $0 \leq k_U(x) - k_A(x) \leq k_U(x)$; разность и симметрическая разность не больше $k_U(x) - k_B(x)$ или $k_U(x) - k_A(x)$; сумма и произведение ограничены явно; $\max\{k_A(x) - k_B(x), 0\} \leq k_A(x)$; $\lfloor k_A(x)/k_B(x) \rfloor \leq k_A(x)$ при $k_B(x) \geq 1$.

Разность, арифметическая разность и деление некоммутативны, дополнение одноместно, поэтому вычисляются оба варианта: $A \setminus B$ и $B \setminus A$, $\bar{A}$ и $\bar{B}$, $A - B$ и $B - A$, A/B и B/A . Вместе с $A \cup B$, $A \cap B$, $A \triangle B$, $A + B$ и $A * B$ это тринадцать результатов.

Для проверки результатов используются следствия определений:

$$|\bar{A}| = |U| - |A|, \quad |A \cup B| + |A \cap B| = |A| + |B|. \quad (9)$$

Второе равенство верно поэлементно: $\max\{a, b\} + \min\{a, b\} = a + b$. Программа проверяет оба равенства сама: после вывода результатов операций (команда `o`, метод `Menu::show_operations`) она вычисляет обе части по полученным мощностям и печатает «верно» или «НЕВЕРНО».

1.4 Универсум на коде Грея

Двоичным словом длины n называют последовательность из n символов 0 и 1 [1], например 011; так принято называть элементы кода, чтобы отличать их от чисел. Носитель X — множество всех 2^n двоичных слов длины n , записанных в порядке кода Грея (подраздел 1.5); элемент универсума с номером i — i -е слово кода. Все элементы универсума имеют одну и ту же кратность k :

$$k_U(x) = k \quad \text{для всех } x \in X, \quad 1 \leq k \leq K_{\max} = 100, \quad |U| = k \cdot 2^n. \quad (10)$$

Кратность k вводит пользователь, либо (пустая строка) программа выбирает её равновероятно из $[1, K_{\max}]$. Подмультимножества такого U — это все A с $0 \leq k_A(x) \leq k$ для каждого элемента.

При $k = 1$ все кратности равны 0 или 1, U и его подмультимножества становятся обычными множествами. Тогда арифметические операции совпадают с теоретико-множественными: $A + B = A \cup B$ (так как $\min\{a + b, 1\} = \max\{a, b\}$ при $a, b \in \{0, 1\}$), $A - B = A \setminus B$, $A * B = A \cap B$, а $A/B = B/A = A \cap B$ ($\lfloor a/b \rfloor = 1$ только при $a = b = 1$). Программа не обрабатывает этот случай

особо: она печатает все тринадцать результатов, и совпадающие строки видны в выводе.

Случай $n = 0$. Двоичное слово длины 0 не содержит ни одного разряда. Такое слово нельзя ни показать в таблице универсума, ни ввести с клавиатуры как код элемента (пустая строка в программе означает отмену ввода). Поэтому в программе принято, что при $n = 0$ носитель пуст: $X = \emptyset$, $U = \{\}$, $|U| = 0$. Из (8) следует, что единственное подмультимножество пустого универсума — пустое, и допустимая мощность A и B равна только 0. Все функции кратности из таблицы 2 определены на пустом носителе, поэтому каждая из тринадцати операций даёт $\{\}$ с мощностью 0; соотношения (9) выполняются: $0 = 0 - 0$ и $0 + 0 = 0 + 0$. Кратность k при $n = 0$ не запрашивается: задавать её некому. Программа проходит этот случай теми же функциями, что и общий (сценарий 5 в подразделе 3.5).

1.5 Алгоритм генерации бинарного кода Грея

Бинарный код Грея порядка n — последовательность всех 2^n двоичных слов длины n , в которой соседние слова, а также последнее и первое, различаются ровно в одном разряде [1].

Алгоритм построения взят из [1]: слово рассматривается как битовая шкала, и на каждом шаге инвертируется один разряд, номер которого определяется номером шага. В [1] разряды нумеруются с 1, поэтому номер разряда там равен числу нулей в конце номера шага, увеличенному на 1. В работе разряды нумеруются с 0, как биты целого числа в программе: $B = (B[n-1], \dots, B[1], B[0])$, $B[0]$ — младший. Тогда прибавка 1 не нужна: на шаге i инвертируется разряд с номером $Q(i)$, где

$$Q(i) = \text{число нулей в конце двоичной записи } i. \quad (11)$$

$Q(i)$ вычисляется так: $q := 0, j := i$; пока j чётно, $j := j/2$ и $q := q + 1$; результат — q . Например, $Q(4) = Q(100_2) = 2$, $Q(6) = Q(110_2) = 1$.

Вход: целое $n \geq 1$.

Выход: последовательность из 2^n шкал длины n .

1. Положить $B[k] := 0$ для $k = 0, \dots, n-1$ и выдать B .

2. Для $i = 1, 2, \dots, 2^n - 1$: $p := Q(i)$; $B[p] := 1 - B[p]$; выдать B .

При $0 < i < 2^n$ двоичная запись i оканчивается не более чем $n-1$

нулями, поэтому $0 \leq Q(i) \leq n-1$ и номер разряда всегда допустим. Протокол работы при $n = 3$ — в таблице 3.

Таблица 3 — Протокол алгоритма при $n = 3$

Шаг i	Двоичная запись i	$p = Q(i)$	Шкала B
—	—	—	000
1	001	0	001
2	010	1	011
3	011	0	010
4	100	2	110
5	101	0	111
6	110	1	101
7	111	0	100

Корректность. Соседние шкалы различаются одним разрядом по построению. Докажем, что все 2^n шкал различны и последняя отличается от первой одним разрядом. Обозначим $S_k = (Q(1), \dots, Q(2^k - 1))$ — последовательность номеров инвертируемых разрядов; она не зависит от n . Для $0 < j < 2^k$ числа $2^k + j$ и j оканчиваются одинаковым числом нулей, а $Q(2^k) = k$; значит,

$$S_{k+1} = (S_k, k, S_k). \quad (12)$$

Индукцией по k отсюда следует, что S_k совпадает со своим обращением (палиндром): $S_1 = (0)$, и обращение (S_k, k, S_k) равно $(S_k^R, k, S_k^R) = (S_k, k, S_k)$.

Пусть при длине k алгоритм выдаёт шкалы $g_1, \dots, g_m$, $m = 2^k$, и утверждение для них верно. При длине $k+1$ первые $m-1$ шагов используют S_k и не трогают старший разряд с номером k : получаются $0g_1, \dots, 0g_m$. Шаг m инвертирует старший разряд: $1g_m$. Оставшиеся $m-1$ шагов снова применяют S_k , то есть, так как S_k — палиндром, обращения S_k , и проходят шкалы в обратном порядке: $1g_{m-1}, \dots, 1g_1$. Итог — $(0g_1, \dots, 0g_m, 1g_m, \dots, 1g_1)$, отражённый код Грея. Шкалы внутри каждой половины различны по предположению индукции, половины различаются старшим разрядом, последняя шкала $1g_1$ отличается от первой $0g_1$ только старшим разрядом. База $k = 1$: шкалы 0 и 1. Полу-

чаемая последовательность совпадает с замкнутой формой $g(i) = i \oplus \lfloor i/2 \rfloor$; программа проверялась на совпадение при $n = 1, \dots, 16$.

Трудоёмкость. Алгоритм делает $2^n - 1$ шагов; на шаге i вычисление $Q(i)$ требует $Q(i) + 1$ проверок чётности (последняя проверка встречает нечётное число). Число $i \leq 2^n - 1$, делящихся на 2^j , равно $\lfloor (2^n - 1)/2^j \rfloor = 2^{n-j} - 1$, поэтому

$$\sum_{i=1}^{2^n-1} (Q(i) + 1) = (2^n - 1) + \sum_{j=1}^n (2^{n-j} - 1) = (2^n - 1) + (2^n - 1 - n) = 2^{n+1} - n - 2,$$

общее время — $O(2^n)$, то есть $O(1)$ в среднем на слово. Память — $O(2^n)$ машинных слов при хранении каждой шкалы целым числом.

1.6 Случайное заполнение мультимножества заданной мощности

Автоматическое заполнение решает задачу: дано мультимножество-ограничение V (в программе это универсум U) и число m , $0 \leq m \leq |V|$; построить случайное $A \subseteq V$ с $|A| = m$.

Элемент x называется *свободным*, если $k_A(x) < k_V(x)$: его кратность в A ещё можно увеличить. Алгоритм хранит список F номеров свободных элементов и m раз увеличивает на 1 кратность случайно выбранного свободного элемента.

Вход: V с кратностями $k_V(x)$; целое m .

Выход: $A \subseteq V$, $|A| = m$.

1. Если $m > |V|$, завершить с ошибкой.
2. Положить $k_A(x) := 0$ для всех x ; занести в список F все x с $k_V(x) > 0$.
3. Повторить m раз: выбрать равномерно позицию j в F , $x := F[j]$; $k_A(x) := k_A(x) + 1$; если $k_A(x) = k_V(x)$, удалить x из F ; записать на место j последний элемент F и укоротить F на 1.
4. Вернуть A .

Корректность. Перед шагом номер s ($s = 0, \dots, m - 1$) в A уже s единиц, поэтому суммарное свободное место $\sum_x (k_V(x) - k_A(x)) = |V| - s \geq |V| - m + 1 > 0$: список F не пуст, и выбор возможен. Каждый шаг добавляет ровно одну единицу, поэтому $|A| = m$. Кратность растёт только у свободных элементов, поэтому $k_A(x) \leq k_V(x)$, то есть $A \subseteq V$. Порядок элементов в F не важен, поэтому удаление за $O(1)$ перестановкой последнего элемента на

место удаляемого корректно.

Распределение. На каждом шаге все свободные элементы равновероятны: вероятность выбрать каждый из них равна $1/|F|$. В программе все кратности универсума равны k , поэтому алгоритм ни одному элементу не отдаёт предпочтения: кратности всех элементов в A распределены одинаково, и так как их сумма равна m ,

$$M k_A(x) = \frac{m}{2^n} \quad \text{для каждого } x. \quad (13)$$

При $m = |V|$ получается $A = V$, при $m = 0$ — $A = \emptyset$.

Трудоёмкость. Построение F — $O(2^n)$, каждый из m шагов — одно случайное число и $O(1)$ действий; холостых выборок уже заполненных элементов нет. Время — $O(2^n + m)$, в худшем случае $m = |U| = k \cdot 2^n$ это $O(K_{\max} \cdot 2^n)$; именно эта величина определяет наибольшую допустимую разрядность (подраздел 2.2).

2 Особенности реализации

2.1 Средства разработки и структура программы

Язык C++14, только стандартная библиотека; компилятор GCC 6.3.0 (MinGW, 32-разрядная сборка), ОС Windows 11. Сборка:

```
g++ -std=c++14 -O2 -Isrc src/main.cpp src/ui/*.cpp  
src/core/*.cpp -o lab1
```

или через make (файл Makefile; в Windows — mingw32-make): make собирает обе программы, make lab1 и make bench — одну, make clean удаляет собранные файлы. Предупреждений при -Wall -Wextra нет.

Код разделён на два слоя: core — математическая часть, не работающая с консолью; ui — ввод и меню. Зависимости направлены только от ui к core. Файлы перечислены в таблице 4, текст программы — в приложении А.

Таблица 4 — Файлы программы

Файл	Назначение
src/core/GrayCode.hpp, .cpp	генерация кода Грея, класс CodeSet (носитель)
src/core/Multiset.hpp, .cpp	класс Multiset, операции, случайный выбор подмультимножества, вывод
src/core/Universe.hpp, .cpp	построение универсума с одинаковыми кратностями
src/core/Random.hpp, .cpp	генератор случайных чисел
src/ui/Input.hpp, .cpp	защищённый построчный ввод
src/ui/Menu.hpp, .cpp	класс Menu: состояние программы и меню
src/main.cpp	точка входа, ключ --seed
tools/bench.cpp	замер времени для выбора наибольшей разрядности

Все величины задачи (разрядность, номера, кратности, мощности, коды) неотрицательны и хранятся в типе `size_t`. В использованной 32-разрядной сборке `size_t` занимает 4 байта (наибольшее значение $2^{32} - 1 = 4\,294\,967\,295$). Наибольшая мощность универсума $100 \cdot 2^{19} = 52\,428\,800$, наибольшее произведение кратностей $100 \cdot 100$, наибольшее вводимое число 999 999 999 — переполнение невозможно. Вычитание беззнаковых величин

выполняется только тогда, когда уменьшаемое заведомо не меньше: в дополнении это гарантирует проверка $A \subseteq U$, в арифметической разности — сравнение $k_A(x) > k_B(x)$, в ручном заполнении — инвариант $k_A(x) \leq k_U(x)$.

2.2 Ограничения разрядности и вывода

Наибольшая разрядность $n_{\max} = 19$ (константа `MAX_DEPTH`). Критерий выбора: любое действие пользователя должно завершаться быстрее 1 с. При большей задержке ответ на команду меню заметно запаздывает, и программа выглядит «зависшей».

Самое долгое действие — случайное заполнение (подраздел 1.6): оно делает $|A| \leq |U| = k \cdot 2^n$ шагов. Остальные действия линейны по 2^n с малой константой. Время измерено утилитой `tools/bench.cpp` в наихудшем случае: $k = K_{\max} = 100$ и $|A| = |U|$, лучший из трёх запусков. Машина: Intel Core i7-13700KF, 32 ГБ ОЗУ; сборка -O2. Результаты — в таблице 5.

Таблица 5 — Время действий, мс (наихудший случай, $K_{\max} = 100$)

n	Код Грея	Универсум	Заполнение A	13 операций
16	0,0	0,0	45	1,0
17	0,0	0,0	95	4,1
18	0,5	0,5	239	7,0
19	1,0	1,0	531	15
20	2,0	2,0	1581	53
21	8,5	6,5	12 368	67

До $n = 19$ время растёт примерно вдвое на разряд, так как вдвое растёт $|U|$. Дальше рост резче: кратности и список свободных элементов перестают помещаться в кэш процессора, а обращения к ним случайны. При $n = 19$ наибольшее время 0,53 с укладывается в 1 с, при $n = 20$ оно уже 1,58 с. Поэтому $n_{\max} = 19$. Память не ограничивает: при $n = 19$ один объект `Multiset` хранит 2^{19} кратностей по 4 байта, то есть 2 097 152 байт (около 2 Мбайт); носитель хранит два таких массива (`codes` и `positions`) — около 4 Мбайт.

Независимо от критерия времени есть предел типа: 2^n должно помещаться в `size_t`, поэтому `generate_gray_code` отвергает $n > 31$ (константа `MAX_TYPE_DEPTH`). `MAX_DEPTH` проверяет меню при вводе n , а `MAX_TYPE_DEPTH` — функция `generate_gray_code`. Утилита замера не

использует меню: она вызывает функции построения кода, универсума и операций напрямую, поэтому ограничение 19 на неё не действует, и она может измерить время при $n = 20$ и $n = 21$, чтобы показать, что там предел в 1 с уже нарушен.

Наибольшая кратность $K_{\max} = 100$ (константа `MAX_MULTPLICITY`). Это ограничение выбрано, а не вытекает из задачи. Оно определяет наибольшую мощность универсума $|U| = K_{\max} \cdot 2^n$, а с ней и время случайного заполнения в худшем случае: при $K_{\max} = 100$ и $n = 19$ оно 0,53 с (таблица 5). Поэтому $K_{\max}$ и $n_{\max}$ связаны: при увеличении $K_{\max}$ замер нужно повторить, и $n_{\max}$ может уменьшиться. Кроме того, при $K_{\max} = 100$ произведение кратностей не больше $100 \cdot 100 = 10\,000$ и не переполняет `size_t`, а каждая кратность записывается не более чем тремя цифрами, на что рассчитана ширина столбцов таблицы и матрицы.

Ограничения вывода. Таблица выводит не больше 32 строк (`TABLE_LIMIT`; при $n \leq 5$ — полностью); список слов кода — не больше 16 (`PRINT_LIMIT`), в строке — столько, сколько помещается в 100 символов; запись мультимножества при $n \leq 4$ — всегда полностью (в ней не больше $2^4 = 16$ элементов). При $n > 4$ коды длинные, и запись ограничена по ширине: элемент вида «00000×100, » занимает $n + 6$ символов, поэтому выводятся первые $\lfloor 100/(n + 6) \rfloor$ элементов, но не меньше 4 (константа `LINE_WIDTH = 100`). Например, при $n = 5$ это $\lfloor 100/11 \rfloor = 9$ элементов, при $n = 19$ — $\lfloor 100/25 \rfloor = 4$; матрица кратностей — столько столбцов кодов, сколько помещается в строку из 112 символов (`MATRIX_WIDTH`; при $n \leq 4$ — все). Дальше выводится пометка, сколько строк, элементов или столбцов показано, а мощности выводятся всегда полностью и считаются по всем элементам. При $n = 19$ полный вывод одного результата занял бы около 2^{19} записей и был бы необозрим.

2.3 Исключения

Ошибки пользователя исключений не вызывают: программа выводит сообщение и повторяет запрос. Исключения сигнализируют о нарушении внутренних ограничений, которое возможно только при ошибке в самой программе, и о конце потока ввода (таблица 6).

Таблица 6 — Классы исключений

Класс	Родитель	Условие
GrayCodeError	<code>std::runtime_error</code>	общий родитель ошибок кода Грея
InvalidDepthError	GrayCodeError	$n > \text{MAX_TYPE_DEPTH}$
CodeIndexError	GrayCodeError	номер кода вне носителя
MultisetError	<code>std::runtime_error</code>	общий родитель ошибок мультимножеств
MultisetIndexError	MultisetError	номер элемента вне носителя
CarrierMismatchError	MultisetError	операнды построены над разными носителями
NotSubmultisetError	MultisetError	операнд не является подмультимножеством U
CardinalityOverflowError	MultisetError	требуемая мощность больше мощности ограничения
InvalidMultiplicityError	MultisetError	$k \notin [1, K_{\max}]$
InputClosedError	<code>std::runtime_error</code>	поток ввода закончился

2.4 Модуль случайных чисел

Источник случайности — генератор `std::mt19937` (вихрь Мерсенна, 32-разрядные значения); равномерность обеспечивает `std::uniform_int_distribution`. Стандартная `std::rand` не подходит: в использованной сборке $\text{RAND_MAX} = 32\,767$, и при $n \geq 16$, когда свободных элементов больше 32 768, часть из них в алгоритме подраздела 1.6 была бы недостижима.

- `unsigned make_seed()` — возвращает зерно: `std::random_device` *исключающее ИЛИ* текущее время. В MinGW `std::random_device` детерминирован (проверено: два запуска выдают одинаковые числа), поэтому без времени все запуски давали бы одни и те же универсумы.
- `void seed_random(unsigned seed)` — задаёт начальное состояние генератора.
- `size_t random_below(size_t bound)` — вход: $\text{bound} > 0$; выход: равновероятное целое из $[0, \text{bound} - 1]$.

Функция `main` выводит использованное зерно; запуск `lab1 --seed N` повторяет все случайные числа. Так получены воспроизводимые сценарии раздела 3.

2.5 Генерация кода Грея и носитель

Шкала B хранится одним целым числом: разряд $B[p]$ — бит числа с номером p (нумерация справа с нуля, как в подразделе 1.5). Функция `changed_bit` вычисляет $Q(i)$ по (11), то есть сразу номер бита, поэтому инвертирование записывается как `code ^= 1 << changed_bit(i)`: число `1 << p` содержит единицу только в бите p , и исключающее ИЛИ с ним меняет именно этот бит. Реализация — в листинге 1.

Листинг 1 — Генерация кода Грея (`GrayCode.cpp`)

```
4 // Функция, определяющая, какой бит перевернуть
5 static size_t changed_bit(size_t i){
6     size_t q = 0;
7     size_t j = i;
8     while(j % 2 == 0){
9         j /= 2;
10        q++;
11    }
12    // число нулей в конце двоичной записи i — номер бита справа, считая с
    нуля
13    return q;
14 }
15
16 std::vector<size_t> generate_gray_code(size_t depth){
17     /*
18        Алгоритм начинает с 0 и на каждом шаге переворачивает
19        один бит, какой бит перевернуть - решает функция changed_bit()
20     */
21     if(depth > MAX_TYPE_DEPTH){
22         throw InvalidDepthError();
23     }
24     std::vector<size_t> codes;
25     // при n = 0 кодов нет
26     if(depth == 0){
27         return codes;
28     }
29     size_t count = size_t(1) << depth;
30     codes.reserve(count);
31     size_t code = 0;
32     codes.push_back(code);
33     for(size_t i = 1; i < count; i++){
```

```

34     size_t p = changed_bit(i);
35     code ^= size_t(1) << p;
36     codes.push_back(code);
37 }
38 return codes;
39 }

```

- `static size_t changed_bit(size_t i)` — вход: $i > 0$; выход: число нулей в конце двоичной записи i , то есть $Q(i)$ по (11). Видна только в `GrayCode.cpp`.
- `std::vector<size_t> generate_gray_code(size_t depth)` — вход: n ; выход: 2^n кодов в порядке построения; при $n = 0$ — пустая последовательность; при $n > 31$ — исключение `InvalidDepthError`.
- `std::string code_to_string(size_t code, size_t depth)` — выход: строка из n символов 0 и 1, старший разряд слева.

Класс `CodeSet` — носитель мультимножеств (таблицы 7, 8). Поле `positions` — обратная таблица: `positions[c]` — номер кода c в последовательности. Так как код Грея содержит все 2^n слов, номер есть у каждого слова длины n , и поиск элемента по введённой строке выполняется за $O(n)$ (перевод строки в число и одно обращение к массиву) вместо $O(n \cdot 2^n)$ при просмотре всех кодов.

Таблица 7 — Поля класса `CodeSet`

Поле	Тип	Назначение
<code>depth</code>	<code>size_t</code>	разрядность n
<code>codes</code>	<code>std::vector<size_t></code>	<code>codes[i]</code> — i -й код Грея
<code>positions</code>	<code>std::vector<size_t></code>	<code>positions[c]</code> — номер кода c

Таблица 8 — Методы класса `CodeSet`

Метод	Вход	Выход
<code>CodeSet(depth)</code>	разрядность n	носитель из 2^n кодов (пустой при $n = 0$)
<code>get_depth()</code>	—	n
<code>size()</code>	—	число элементов 2^n (0 при $n = 0$)
<code>get_code(i)</code>	номер i	строка кода; <code>CodeIndexError</code> при $i \geq 2^n$
<code>find(code, index)</code>	строка; переменная для номера	<code>true</code> и номер в <code>index</code> , если строка — слово длины n из 0 и 1; иначе <code>false</code>

2.6 Класс `Multiset`

Класс `Multiset` — мультимножество над носителем, представленное функцией кратности. Объектами этого класса являются и универсум U , и мультимножества A , B , и все результаты операций: универсум отличается только тем, что его кратности не меньше 1 и он передаётся операциям, которым нужна $k_U(x)$. Поля — в таблице 9.

Таблица 9 — Поля класса `Multiset`

Поле	Тип	Назначение
<code>carrier</code>	<code>std::shared_ptr<const CodeSet></code>	общий носитель; объект носителя живёт, пока на него ссылается хотя бы одно мультимножество
<code>counts</code>	<code>std::vector<size_t></code>	<code>counts[i]</code> — кратность i -го кода Грея

Кратности хранятся по номерам элементов носителя, поэтому поиск элементов в операциях не нужен, и каждая операция выполняется за $O(2^n)$ одним проходом (разность — двумя, симметрическая разность — пятью, так как они собраны из других операций). Совместное владение носителем через `shared_ptr` исключает висячие указатели: при пересоздании универсума старые мультимножества продолжают ссылаться на свой носитель. Принадлежность одному носителю проверяется сравнением указателей.

Методы доступа — в таблице 10. Закрытый метод `check_same_carrier` вызывается каждой двуместной операцией, `check_within` — каждой операцией, которой передаётся универсум.

Таблица 10 — Методы доступа класса `Multiset`

Метод	Вход	Выход
<code>Multiset(carrier)</code>	носитель	мультимножество с нулевыми кратностями
<code>size()</code>	—	число элементов носителя
<code>get_count(i)</code>	номер	кратность; <code>MultisetIndexError</code>
<code>set_count(i, count)</code>	номер, кратность	кратность изменена; <code>MultisetIndexError</code>
<code>cardinality()</code>	—	мощность по (7)
<code>is_submultiset_of(other)</code>	мультимножество	истина, если выполнено (8)
<code>check_same_carrier(other)</code>	мультимножество	<code>CarrierMismatchError</code> , если носители разные
<code>check_within(universe)</code>	универсум	<code>NotSubmultisetError</code> , если объект не подмультимножество U (носители сравнивает вызываемый <code>is_submultiset_of</code>)

Методы операций (таблица 11): первый операнд A — сам объект, второй B — параметр; каждый метод возвращает новый объект и не меняет операнды. Метод объединения назван `unite`, так как `union` — ключевое слово C++.

Таблица 11 — Методы операций класса Multiset

Метод	Результат	Реализация
<code>unite(b)</code>	$A \cup B$	max по элементам
<code>intersect(b)</code>	$A \cap B$	min по элементам
<code>complement(u)</code>	$\bar{A}$	$k_U(x) - k_A(x)$ после проверки $A \subseteq U$
<code>difference(b, u)</code>	$A \setminus B$	$A \cap \bar{B}$ методами <code>intersect</code> и <code>complement</code>
<code>symmetric_difference(b, u)</code>	$A \triangle B$	$(A \setminus B) \cup (B \setminus A)$ методами <code>difference</code> и <code>unite</code>
<code>arithmetic_sum(b, u)</code>	$A + B$	$\min\{k_A(x) + k_B(x), k_U(x)\}$
<code>arithmetic_difference(b)</code>	$A - B$	$k_A(x) - k_B(x)$ при $k_A(x) > k_B(x)$, иначе 0
<code>arithmetic_product(b, u)</code>	$A * B$	$\min\{k_A(x) \cdot k_B(x), k_U(x)\}$
<code>arithmetic_division(b)</code>	A / B	$\lfloor k_A(x) / k_B(x) \rfloor$ при $k_B(x) \neq 0$, иначе 0

Функция `random_submultiset(bound, cardinality)` реализует алгоритм подраздела 1.6 (листинг 2); вход: ограничение V и мощность m ; выход: случайное $A \subseteq V$, $|A| = m$; при $m > |V|$ — `CardinalityOverflowError`. Список свободных элементов — `std::vector<size_t> free`; удаление заполненного элемента — `free[j] = free.back(); free.pop_back();`. Функция объявлена другом класса, чтобы работать с `counts` и `carrier` напрямую.

Листинг 2 — Случайное заполнение мультимножества (`Multiset.cpp`)

```

135 Multiset random_submultiset(const Multiset& bound, size_t cardinality){
136     if(cardinality > bound.cardinality()){
137         throw CardinalityOverflowError();
138     }
139     Multiset res(bound.carrier);
140     // номера элементов, кратность которых в res ещё меньше, чем в bound
141     std::vector<size_t> free;
142     free.reserve(bound.counts.size());
143     for(size_t i = 0; i < bound.counts.size(); i++){
144         if(bound.counts[i] > 0) free.push_back(i);
145     }
146     // cardinality раз: случайный элемент со свободным местом получает +1 к
    кратности;
147     // список не пустеет раньше времени, так как свободных мест |bound| -
    step > 0
148     for(size_t step = 0; step < cardinality; step++){

```

```

149     size_t j = random_below(free.size());
150     size_t i = free[j];
151     res.counts[i]++;
152     if(res.counts[i] == bound.counts[i]){
153         // элемент заполнен: на его место в списке встаёт последний,
154         // список короче на 1
155         free[j] = free.back();
156         free.pop_back();
157     }
158     return res;
159 }

```

Вывод: `print_multiset(out, multiset, limit)` записывает в поток элементы с ненулевой кратностью в виде `{000×2, 011×1}` (код Грея и через знак $\times$ его кратность), не больше `limit` элементов, и пометку «... ещё N эл.»; пустое мультимножество записывается как `{ }`. Оператор `<<` выводит мультимножество полностью.

Деструкторы, конструкторы копирования и перемещения не объявлены явно: классы содержат только `std::vector`, `std::shared_ptr` и числа, и сгенерированные компилятором функции корректны.

2.7 Построение универсума

Модуль `Universe` содержит константу `MAX_MULTIPPLICITY =  $K_{\max} = 100$` и функцию

— `Multiset uniform_universe(carrier, multiplicity)` —
 вход: носитель и $k \in [1, K_{\max}]$; выход: универсум, в котором все 2^n элементов имеют кратность k ; иначе `InvalidMultiplicityError`.

Случайную кратность выбирает меню: $k = 1 + \text{random_below}(K_{\max})$, то есть равномерно из $[1, K_{\max}]$.

2.8 Модуль ввода

Весь ввод читается построчно функцией `std::getline` и проверяется целиком. Оператор `>>` не подходит: он принимает «12abc» как 12, превращает «-1» в большое беззнаковое число и после ошибки оставляет поток в состоянии сбоя, из-за чего программа заикливается.

— `std::string read_text(prompt)` — печатает приглашение, читает строку, убирает пробелы, табуляции и `\r` по краям; при конце потока —

`InputClosedError`. Если ввод перенаправлен из файла (стандартный ввод не консоль), функция выводит прочитанную строку, чтобы протокол выглядел как сеанс в консоли. Открыта, так как строку ручного заполнения разбирает меню.

- `bool parse_number(text, value)` — истина, если строка состоит из 1–9 цифр; число записывается в `value`. Отвергаются знаки, точки, пробелы внутри и пустая строка.
- `bool is_code(text, depth)` — истина, если строка длины n из символов 0 и 1.
- `size_t read_number(prompt, min, max)` — повторяет запрос, пока не получит число из `[min, max]`. Сообщение называет причину отказа: «пустая строка: нужно целое число из `[min..max]`», « x не входит в `[min..max]`» (число вне отрезка) или «« s » не целое неотрицательное число; нужно число из `[min..max]`».
- `bool read_number_or_empty(prompt, min, max, value)` — то же, но на пустую строку возвращает `false`.
- `char read_letter(prompt, letters, allow_empty)` — повторяет запрос, пока не получит ровно одну букву из `letters`; заглавная латинская буква приводится к строчной. Если `allow_empty`, пустая строка допустима и возвращается `'\0'`. Сообщения: «пустая строка: введите одну из букв ...» и «« s » не команда; допустимы буквы ...».

2.9 Класс Menu и функция main

Класс `Menu` хранит состояние программы: общий носитель `carrier`, мультимножества `universe`, `a`, `b` и признаки `universe_ready`, `a_ready`, `b_ready`. Признак создания универсума нужен отдельно от его пустоты: при $n = 0$ универсум создан, но пуст.

Интерфейс минималистичный и построен на однобуквенных командах (таблица 12). Список команд выводится одной строкой при запуске и в справке; приглашение к вводу команды — «>», действия отделяются пустой строкой. Рамок, заголовков и псевдографики нет: действие выводит только запросы, сообщения об ошибках (начинаются с «!») и результат.

Таблица 12 — Команды меню

Буква	Действие	Метод	Условие
u	создать универсум	<code>create_universe</code>	—
a, b	задать A, B	<code>fill_multiset</code>	U создан
t	таблица кратностей U, A, B	<code>show_table</code>	U создан
o	тринадцать операций и проверка мощностей	<code>show_operations</code>	заданы A и B
m	матрица кратностей: мультимножество $\times$ код	<code>show_matrix</code>	заданы A и B
h	справка: команды, запись, формулы операций, ограничения	<code>show_help</code>	—
q	выход	—	—

Если условие команды не выполнено, `is_available` называет, чего именно не хватает, например «команда o недоступна: не задано A (команда a) и B (команда b)», и команда не выполняется.

Названия, обозначения и формулы кратности тринадцати операций записаны один раз — в массиве `OPERATIONS` из структур `OperationInfo`; `compute_operations` возвращает результаты в том же порядке. По этому массиву строятся справка, список операций и строки матрицы.

Создание универсума (`create_universe`): читается $n \in [0, 19]$ и строится новый носитель. При $n = 0$ выводится «при $n = 0$ в коде Грея нет слов, кратность не нужна», и кратность не запрашивается. Иначе читается $k \in [1, 100]$; на пустую строку k выбирается случайно и выводится. Универсум строится функцией `uniform_universe`. Новые объекты собираются в локальных переменных и присваиваются полям только в конце. После создания A и B становятся пустыми мультимножествами над новым носителем со снятыми признаками. Затем выводятся число слов 2^n и первые слова кода Грея (не больше 16), запись U и $|U| = 2^n \cdot k$; если A и B были заданы, выводится « A и B сброшены».

Заполнение (`fill_multiset`): сначала читается мощность $m \in [0, |U|]$; на пустую строку m выбирается равновероятно из $[0, |U|]$ и выводится. Затем читается способ: r — случайное, m — ручное, пустая строка — отмена. Заполняется новый объект; поле и признак обновляются только при успехе. Случайное заполнение — `random_submultiset(universe, m)`.

Ручное заполнение (`fill_manual`, `parse_manual_line`). Пользователь вводит строку пар код:кратность через пробел; код без двоеточия означает кратность 1, например 01:2 11.

1. Если $m = 0$, вводить нечего: завершить, $A = \{\}$.
2. Пока $|A| < m$: вывести в приглашении текущее A и остаток $d = m - |A|$; прочитать строку (пустая — отмена всего заполнения).
3. Для каждой пары строки по порядку: код должен состоять из n символов 0 и 1; после двоеточия должно стоять целое $c \geq 1$; c не больше свободного места кода $f = k_U(x) - k_A(x) - s$, где s — единицы этого же кода, уже взятые ранее в той же строке.
4. Сумма c по строке не должна превышать d .
5. Если все проверки пройдены, прибавить все пары к A ; иначе вывести причину первой ошибки (например, ««00:3»: у 00 свободно 2 ($k_U = 2$, занято 0)» или «в строке 5 ед., а осталось распределить 4») и не менять A . Вернуться к шагу 2.

Строка применяется целиком или не применяется вовсе, поэтому после любой ошибки A остаётся в состоянии до этой строки. Проверки шагов 3 и 4 сохраняют $A \subseteq U$ и $|A| \leq m$; так как $m \leq |U|$, свободное место есть всегда, пока $|A| < m$.

Вывод.

- `show_table`: столбцы «код», « k_U », « k_A », « k_B » (для незаданного мультимножества — «-»). Последняя строка « Σ » — мощности. Ширина числовых столбцов — по длине записи $|U|$.
- `show_operations`: по строке на операцию — обозначение, запись результата и его мощность (сначала семь теоретико-множественных, затем шесть арифметических). В конце две строки «проверка» вычисляют соотношения (9) по полученным результатам и печатают «верно» или «НЕВЕРНО».
- `show_matrix`: строка — мультимножество (U, A, B и тринадцать результатов), столбец — код Грея, последний столбец « $|\cdot|$ » — мощность. Ручная проверка любой операции для конкретного кода сводится к чтению одного столбца.
- `show_help`: список команд, запись мультимножества, таблица операций из OPERATIONS (обозначение, формула кратности, название) и ограниче-

ния входных данных.

Ширины столбцов считаются в символах, а не байтах: строки в UTF-8, и такие символы, как «U» или «Σ», занимают несколько байт.

Функция `main` в Windows вызывает `SetConsoleOutputCP(CP_UTF8)`, так как консоль по умолчанию выводит в кодировке 866; задаёт зерно (`--seed N` или `make_seed`) и печатает его; запускает меню. `InputClosedError` обрабатывается как штатное завершение: «ввод закончился», код возврата 0. Любое другое исключение (возможно только при ошибке в программе) — сообщение и код возврата 1.

3 Результаты работы

Работа проверена на семи сценариях. Ввод каждого сценария записан в файл `report/scenarios/*.txt`, и программа запускалась командой `lab1 --seed N < файл`, где N — номер сценария; при повторном запуске вывод совпадает до символа. Так как ввод берётся из файла, введённые строки печатает сама программа (подраздел 2.8), и протокол совпадает с тем, что видно в консоли при ручном вводе. Рисунки получены из этих протоколов скриптом `report/make_figures.py`: каждый рисунок начинается со строки ввода команды после приглашения «>»; если рисунок охватывает несколько действий, они разделены пустой строкой.

Кроме сценариев, ядро проверено отдельной программой: для $n = 0, \dots, 10$ и 200 случайных пар A, B на каждое n все тринадцать результатов совпали с формулами таблицы 2 поэлементно и оказались подмультимножествами U ; для $n = 1, \dots, 16$ код Грея проверен на цикличность, различие соседних слов в одном разряде и совпадение с $i \oplus \lfloor i/2 \rfloor$; для ограничений с разными кратностями проверено, что результат заполнения имеет ровно заданную мощность и вложен в ограничение; доля выбора элемента x при $V = \{x^2, y^1\}$, $m = 1$ составила 0,5002 на 300 000 испытаний при теоретической $1/2$ (свободные элементы равновероятны).

3.1 Сценарий 1: случайная кратность, автоматическое заполнение

После запуска программа печатает зерно и строку со списком команд (рисунок 1). Команда `h` выводит справку: список команд, запись мультимножества, формулы кратности всех тринадцати операций и ограничения ввода (рисунок 2).

```
seed = 1
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
> h
```

Рисунок 1 — Запуск программы, зерно 1

```

> h
u - универсум, a/b - задать A/B, t - таблица, o - операции, m - матрица, h - справка, q - выход
запись {000×2, 011×1}: код Грея × кратность, {} - пустое; kU, kA, kB - кратности в U, A, B
A ∪ B max(kA, kB) объединение
A ∩ B min(kA, kB) пересечение
A \ B min(kA, kU - kB) разность
B \ A min(kB, kU - kA) разность
A Δ B max(k[A\B], k[B\A]) симметрическая разность
¬A kU - kA дополнение
¬B kU - kB дополнение
A + B min(kA + kB, kU) арифметическая сумма
A - B max(kA - kB, 0) арифметическая разность
B - A max(kB - kA, 0) арифметическая разность
A * B min(kA * kB, kU) арифметическое произведение
A / B kA div kB; 0 при kB = 0 арифметическое деление
B / A kB div kA; 0 при kA = 0 арифметическое деление
0 <= n <= 19, 1 <= k <= 100, 0 <= |A|, |B| <= |U|

```

Рисунок 2 — Справка (команда h)

Создаётся универсум с $n = 3$; кратность не вводится (Enter), и программа выбирает её случайно: $k = 42$. Выводятся восемь слов кода в порядке таблицы 3, запись U и $|U| = 8 \cdot 42 = 336$ (рисунок 3).

```

> u
n [0..19]: 3
k [1..100, Enter - случайно]:
k = 42
код Грея, 2^3 = 8: 000 001 011 010 110 111 101 100
U = {000×42, 001×42, 011×42, 010×42, 110×42, 111×42, 101×42, 100×42}, |U| = 336

```

Рисунок 3 — Создание универсума: $n = 3$, случайная кратность $k = 42$

A заполняется случайно с заданной мощностью 150; для B мощность не вводится и выбирается случайно: $|B| = 219$ (рисунок 4). Таблица команды t (рисунок 5) подтверждает $k_A(x) \leq 42$ и $k_B(x) \leq 42$ для всех восьми элементов, а строка « Σ » — мощности $|A| = 150$ и $|B| = 219$.

```

> a
|A| [0..336, Enter - случайно]: 150
r - случайно, m - вручную, Enter - отмена: r
A = {000×21, 001×20, 011×14, 010×21, 110×21, 111×16, 101×14, 100×23}, |A| = 150

> b
|B| [0..336, Enter - случайно]:
|B| = 219
r - случайно, m - вручную, Enter - отмена: r
B = {000×37, 001×30, 011×28, 010×20, 110×31, 111×20, 101×28, 100×25}, |B| = 219

```

Рисунок 4 — Случайное заполнение: $|A|$ задана, $|B|$ выбрана случайно

> t			
код	kU	kA	kB
000	42	21	37
001	42	20	30
011	42	14	28
010	42	21	20
110	42	21	31
111	42	16	20
101	42	14	28
100	42	23	25
Σ	336	150	219

Рисунок 5 — Таблица кратностей U, A, B

Результаты операций — на рисунке 6, матрица кратностей — на рисунке 7. Две строки «проверка» в конце рисунка 6 подтверждает соотношения (9): $|\bar{A}| = 336 - 150 = 186$, $|A \cup B| + |A \cap B| = 220 + 149 = 369 = 150 + 219$.

```
> o
A ∪ B = {000×37, 001×30, 011×28, 010×21, 110×31, 111×20, 101×28, 100×25}, |A ∪ B| = 220
A ∩ B = {000×21, 001×20, 011×14, 010×20, 110×21, 111×16, 101×14, 100×23}, |A ∩ B| = 149
A \ B = {000×5, 001×12, 011×14, 010×21, 110×11, 111×16, 101×14, 100×17}, |A \ B| = 110
B \ A = {000×21, 001×22, 011×28, 010×20, 110×21, 111×20, 101×28, 100×19}, |B \ A| = 179
A Δ B = {000×21, 001×22, 011×28, 010×21, 110×21, 111×20, 101×28, 100×19}, |A Δ B| = 180
¬A    = {000×21, 001×22, 011×28, 010×21, 110×21, 111×26, 101×28, 100×19}, |¬A| = 186
¬B    = {000×5, 001×12, 011×14, 010×22, 110×11, 111×22, 101×14, 100×17}, |¬B| = 117
A + B = {000×42, 001×42, 011×42, 010×41, 110×42, 111×36, 101×42, 100×42}, |A + B| = 329
A - B = {010×1}, |A - B| = 1
B - A = {000×16, 001×10, 011×14, 110×10, 111×4, 101×14, 100×2}, |B - A| = 70
A * B = {000×42, 001×42, 011×42, 010×42, 110×42, 111×42, 101×42, 100×42}, |A * B| = 336
A / B = {010×1}, |A / B| = 1
B / A = {000×1, 001×1, 011×2, 110×1, 111×1, 101×2, 100×1}, |B / A| = 9
проверка: |¬A| = |U| - |A|: 186 = 336 - 150 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 220 + 149 = 150 + 219 верно
```

Рисунок 6 — Результаты тринадцати операций (сценарий 1)

> m	000	001	011	010	110	111	101	100	·
U	42	42	42	42	42	42	42	42	336
A	21	20	14	21	21	16	14	23	150
B	37	30	28	20	31	20	28	25	219
$A \cup B$	37	30	28	21	31	20	28	25	220
$A \cap B$	21	20	14	20	21	16	14	23	149
$A \setminus B$	5	12	14	21	11	16	14	17	110
$B \setminus A$	21	22	28	20	21	20	28	19	179
$A \Delta B$	21	22	28	21	21	20	28	19	180
$\neg A$	21	22	28	21	21	26	28	19	186
$\neg B$	5	12	14	22	11	22	14	17	117
$A+B$	42	42	42	41	42	36	42	42	329
$A-B$	0	0	0	1	0	0	0	0	1
$B-A$	16	10	14	0	10	4	14	2	70
$A*B$	42	42	42	42	42	42	42	42	336
A/B	0	0	0	1	0	0	0	0	1
B/A	1	1	2	0	1	1	2	1	9

Рисунок 7 — Матрица кратностей (сценарий 1)

Ручной расчёт для трёх элементов по таблице 2 приведён в таблице 13 и совпадает со столбцами 000, 010 и 100 матрицы на рисунке 7.

Таблица 13 — Ручная проверка (сценарий 1, $k_U = 42$)

	000: $k_A=21, k_B=37$	010: $k_A=21, k_B=20$	100: $k_A=23, k_B=25$
$A \cup B, A \cap B$	37, 21	21, 20	25, 23
$A \setminus B$	$\min\{21, 5\} = 5$	$\min\{21, 22\} = 21$	$\min\{23, 17\} = 17$
$B \setminus A$	$\min\{37, 21\} = 21$	$\min\{20, 21\} = 20$	$\min\{25, 19\} = 19$
$A \Delta B$	$\max\{5, 21\} = 21$	$\max\{21, 20\} = 21$	$\max\{17, 19\} = 19$
$\overline{A}, \overline{B}$	21, 5	21, 22	19, 17
$A+B$	$\min\{58, 42\} = 42$	$\min\{41, 42\} = 41$	$\min\{48, 42\} = 42$
$A-B, B-A$	0, 16	1, 0	0, 2
$A*B$	$\min\{777, 42\} = 42$	$\min\{420, 42\} = 42$	$\min\{575, 42\} = 42$
$A/B, B/A$	$\lfloor 21/37 \rfloor = 0,$ $\lfloor 37/21 \rfloor = 1$	1, 0	0, 1

Элемент 010 показывает различие разностей: $k_{A \setminus B} = \min\{21, 42 - 20\} = 21$, а $k_{A-B} = 21 - 20 = 1$. Все три элемента показывают ограничение универсумом: без него k_{A*B} было бы 777, 420 и 575, программа выводит 42; сумма для 000 и 100 тоже ограничена: 58 и 48 превращаются в 42.

3.2 Сценарий 2: ручной ввод с ошибками и отмена

Создаётся универсум с $n = 2$ (рисунок 8). Как кратность вводятся 0, 101 и «abc»: первые два значения отвергаются как числа вне $[1..100]$, третье — как не число. Принимается $k = 2$, $|U| = 8$.

```
> u
n [0..19]: 2
k [1..100, Enter - случайно]: 0
! 0 не входит в [1..100]
k [1..100, Enter - случайно]: 101
! 101 не входит в [1..100]
k [1..100, Enter - случайно]: abc
! «abc» не целое неотрицательное число; нужно число из [1..100]
k [1..100, Enter - случайно]: 2
код Грея,  $2^2 = 4$ : 00 01 11 10
U = {00×2, 01×2, 11×2, 10×2}, |U| = 8
```

Рисунок 8 — Ввод кратности универсума с ошибками

A заполняется вручную с мощностью 4 (рисунок 9). Перед каждой строкой программа показывает текущее A и остаток. Отвергаются:

- «0» и «0a:1» — код не из двух символов 0 и 1;
- «00:3» — у 00 свободно только 2 единицы;
- «00:x» — после двоеточия не число;
- «00:2 01 10 11» — каждая пара допустима, но в строке 5 единиц при остатке 4. Строка не применяется целиком, поэтому 00 не получает кратность 2 «наполовину»: A остаётся {}.

Затем «00:2» принимается, повторный «00» отвергается (свободно 0, занято 2), «11:2» завершает заполнение: $A = \{00^2, 11^2\}$.

```

> a
|A| [0..8, Enter - случайно]: 4
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
A = {}, осталось 4: 0
! «0»: код должен состоять из 2 символов 0 и 1
A = {}, осталось 4: 0a:1
! «0a:1»: код должен состоять из 2 символов 0 и 1
A = {}, осталось 4: 00:3
! «00:3»: у 00 свободно 2 (kU = 2, занято 0)
A = {}, осталось 4: 00:x
! «00:x»: после двоеточия нужна кратность - целое число от 1
A = {}, осталось 4: 00:2 01 10 11
! в строке 5 ед., а осталось распределить 4
A = {}, осталось 4: 00:2
A = {00×2}, осталось 2: 00
! «00»: у 00 свободно 0 (kU = 2, занято 2)
A = {00×2}, осталось 2: 11:2
A = {00×2, 11×2}, |A| = 4

```

Рисунок 9 — Ручное заполнение A с ошибками ввода

Для B мощность не вводится (случайно $|B| = 3$), выбирается ручной способ, и на первом же запросе вводится пустая строка: заполнение отменяется («отменено»), и B остаётся незадаанным. Затем B заполняется случайно с мощностью 3: $B = \{00^2, 11^1\}$ (рисунок 10).

```

> b
|B| [0..8, Enter - случайно]:
|B| = 3
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
B = {}, осталось 3:
отменено

> b
|B| [0..8, Enter - случайно]: 3
r - случайно, m - вручную, Enter - отмена: r
B = {00×2, 11×1}, |B| = 3

```

Рисунок 10 — Отмена ручного заполнения B и случайное заполнение

Результаты — на рисунках 11 и 12. Для 00 ($k_U = k_A = k_B = 2$): $k_{A \setminus B} = \min\{2, 0\} = 0$, хотя элемент есть в A , — он целиком «занят» в B . Для 11 ($k_U = 2, k_A = 2, k_B = 1$): $k_{A \setminus B} = \min\{2, 1\} = 1, k_{A-B} = 1, k_{A+B} = \min\{3, 2\} = 2, k_{A/B} = 2, k_{B/A} = 0$.

```

> o
A ∪ B = {00×2, 11×2}, |A ∪ B| = 4
A ∩ B = {00×2, 11×1}, |A ∩ B| = 3
A \ B = {11×1}, |A \ B| = 1
B \ A = {}, |B \ A| = 0
A Δ B = {11×1}, |A Δ B| = 1
¬A = {01×2, 10×2}, |¬A| = 4
¬B = {01×2, 11×1, 10×2}, |¬B| = 5
A + B = {00×2, 11×2}, |A + B| = 4
A - B = {11×1}, |A - B| = 1
B - A = {}, |B - A| = 0
A * B = {00×2, 11×2}, |A * B| = 4
A / B = {00×1, 11×2}, |A / B| = 3
B / A = {00×1}, |B / A| = 1
проверка: |¬A| = |U| - |A|: 4 = 8 - 4 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 4 + 3 = 4 + 3 верно

```

Рисунок 11 — Результаты операций (сценарий 2)

```

> m

```

	00	01	11	10	·
U	2	2	2	2	8
A	2	0	2	0	4
B	2	0	1	0	3
A∪B	2	0	2	0	4
A∩B	2	0	1	0	3
A\B	0	0	1	0	1
B\A	0	0	0	0	0
AΔB	0	0	1	0	1
¬A	0	2	0	2	4
¬B	0	2	1	2	5
A+B	2	0	2	0	4
A-B	0	0	1	0	1
B-A	0	0	0	0	0
A*B	2	0	2	0	4
A/B	1	0	2	0	3
B/A	1	0	0	0	1

Рисунок 12 — Матрица кратностей (сценарий 2)

3.3 Сценарий 3: граничные мощности A и B

Создаётся универсум с $n = 3$ и $k = 2$: $|U| = 16$ (рисунок 13).

```

> u
n [0..19]: 3
k [1..100, Enter - случайно]: 2
код Грея, 2^3 = 8: 000 001 011 010 110 111 101 100
U = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |U| = 16

```

Рисунок 13 — Универсум $n = 3, k = 2$

Мощность $|A| = 17$ отвергается сообщением «17 не входит в $[0..16]$ »; при $|A| = 16 = |U|$ получается $A = U$. Для B задана мощность 0: $B = \{\}$ (рисунок 14).

```
> a
|A| [0..16, Enter - случайно]: 17
! 17 не входит в [0..16]
|A| [0..16, Enter - случайно]: 16
r - случайно, m - вручную, Enter - отмена: r
A = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A| = 16

> b
|B| [0..16, Enter - случайно]: 0
r - случайно, m - вручную, Enter - отмена: r
B = {}, |B| = 0
```

Рисунок 14 — Граничные мощности: $A = U, B = \emptyset$

Результаты (рисунки 15, 16) совпадают с ожидаемыми для $A = U, B = \emptyset$: $A \cup B = A \setminus B = A \triangle B = \bar{B} = A + B = A - B = U$; $A \cap B = B \setminus A = \bar{A} = B - A = A * B = \emptyset$. Оба частных пусты: A/B — так как все $k_B = 0$, B/A — так как $\lfloor 0/2 \rfloor = 0$.

```
> o
A ∪ B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A ∪ B| = 16
A ∩ B = {}, |A ∩ B| = 0
A \ B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A \ B| = 16
B \ A = {}, |B \ A| = 0
A Δ B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A Δ B| = 16
¬A = {}, |¬A| = 0
¬B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |¬B| = 16
A + B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A + B| = 16
A - B = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |A - B| = 16
B - A = {}, |B - A| = 0
A * B = {}, |A * B| = 0
A / B = {}, |A / B| = 0
B / A = {}, |B / A| = 0
проверка: |¬A| = |U| - |A|: 0 = 16 - 16 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 16 + 0 = 16 + 0 верно
```

Рисунок 15 — Результаты операций при $A = U, B = \emptyset$

> m	000	001	011	010	110	111	101	100	·
U	2	2	2	2	2	2	2	2	16
A	2	2	2	2	2	2	2	2	16
B	0	0	0	0	0	0	0	0	0
$A \cup B$	2	2	2	2	2	2	2	2	16
$A \cap B$	0	0	0	0	0	0	0	0	0
$A \setminus B$	2	2	2	2	2	2	2	2	16
$B \setminus A$	0	0	0	0	0	0	0	0	0
$A \Delta B$	2	2	2	2	2	2	2	2	16
$\neg A$	0	0	0	0	0	0	0	0	0
$\neg B$	2	2	2	2	2	2	2	2	16
$A + B$	2	2	2	2	2	2	2	2	16
$A - B$	2	2	2	2	2	2	2	2	16
$B - A$	0	0	0	0	0	0	0	0	0
$A * B$	0	0	0	0	0	0	0	0	0
A / B	0	0	0	0	0	0	0	0	0
B / A	0	0	0	0	0	0	0	0	0

Рисунок 16 — Матрица кратностей при $A = U$, $B = \emptyset$

3.4 Сценарий 4: некорректный ввод команд и параметров универсума

До создания универсума вводятся команды а и о; программа сообщает, что универсум ещё не создан, и называет команду, которая его создаёт (рисунок 17).

```
> а
! команда а недоступна: универсум ещё не создан (команда u)

> о
! команда о недоступна: универсум ещё не создан (команда u)
```

Рисунок 17 — Команды а и о до создания универсума

Как команда вводятся «abc», «7», пустая строка, «uu» и «-1»; каждая строка отвергается, и в сообщении перечислены допустимые буквы. Заглавная «U» принимается как команда u (рисунок 18).

```

> abc
! «abc» не команда; допустимы буквы u a b t o m h q
> 7
! «7» не команда; допустимы буквы u a b t o m h q
>
! пустая строка: введите одну из букв u a b t o m h q
> uu
! «uu» не команда; допустимы буквы u a b t o m h q
> -1
! «-1» не команда; допустимы буквы u a b t o m h q
> U

```

Рисунок 18 — Некорректный ввод команды

Как разрядность вводятся 20, «-2» и «abc». Число 20 отвергается как выходящее за $[0..19]$, «-2» и «abc» — как не являющиеся целыми неотрицательными числами. Как кратность вводятся 0 и 101: оба вне $[1..100]$ (рисунок 19). При $n = 3, k = 1$ все кратности равны 1: универсум — обычное множество, $|U| = 8$.

```

n [0..19]: 20
! 20 не входит в [0..19]
n [0..19]: -2
! «-2» не целое неотрицательное число; нужно число из [0..19]
n [0..19]: abc
! «abc» не целое неотрицательное число; нужно число из [0..19]
n [0..19]: 3
k [1..100, Enter - случайно]: 0
! 0 не входит в [1..100]
k [1..100, Enter - случайно]: 101
! 101 не входит в [1..100]
k [1..100, Enter - случайно]: 1
код Грея,  $2^3 = 8$ : 000 001 011 010 110 111 101 100
U = {000×1, 001×1, 011×1, 010×1, 110×1, 111×1, 101×1, 100×1}, |U| = 8

```

Рисунок 19 — Некорректный ввод разрядности и кратности; универсум при $k = 1$

Команда $\circ$ при незаданных A и B называет оба недостающих мультимножества. При заполнении A с мощностью 3 способ «z» отвергается, а пустая строка отменяет заполнение (рисунок 20).

```

> o
! команда o недоступна: не задано A (команда a) и B (команда b)

> a
|A| [0..8, Enter - случайно]: 3
r - случайно, m - вручную, Enter - отмена: z
! «z» не команда; допустимы буквы r m
r - случайно, m - вручную, Enter - отмена:
отменено

```

Рисунок 20 — Команда o до заполнения и отмена заполнения A

3.5 Сценарий 5: разрядность $n = 0$

При $n = 0$ программа сообщает, что в коде Грея нет ни одного слова, выводит $U = \{\}$, $|U| = 0$ и не запрашивает кратность (рисунок 21). Универсум при этом считается созданным: команды a, b и t становятся доступны.

```

> u
n [0..19]: 0
при n = 0 в коде Грея нет слов, кратность не нужна
U = {}, |U| = 0

```

Рисунок 21 — Создание универсума при $n = 0$

A заполняется вручную: мощность 1 отвергается (допустимый отрезок $[0..0]$), при мощности 0 вводить нечего, и строка пар не запрашивается: $A = \{\}$. B заполняется случайно со случайной мощностью; единственное возможное значение — 0, $B = \{\}$ (рисунок 22).

```

> a
|A| [0..0, Enter - случайно]: 1
! 1 не входит в [0..0]
|A| [0..0, Enter - случайно]: 0
r - случайно, m - вручную, Enter - отмена: m
A = {}, |A| = 0

> b
|B| [0..0, Enter - случайно]:
|B| = 0
r - случайно, m - вручную, Enter - отмена: r
B = {}, |B| = 0

```

Рисунок 22 — Ручное и случайное заполнение при $n = 0$

Таблица команды t содержит только строку «Σ» с мощностями $|U| = |A| = |B| = 0$ (рисунок 23). Все тринадцать результатов равны $\{\}$ с мощностью 0,

проверка выполняется: $0 = 0 - 0$ и $0 + 0 = 0 + 0$ (рисунок 24). В матрице нет ни одного столбца кодов, только столбец мощностей из нулей (рисунок 25). Это совпадает с подразделом 1.4.

```
> t
код  kU  kA  kB
Σ    0   0   0
```

Рисунок 23 — Таблица кратностей при $n = 0$

```
> o
A ∪ B = {}, |A ∪ B| = 0
A ∩ B = {}, |A ∩ B| = 0
A \ B = {}, |A \ B| = 0
B \ A = {}, |B \ A| = 0
A Δ B = {}, |A Δ B| = 0
¬A      = {}, |¬A| = 0
¬B      = {}, |¬B| = 0
A + B = {}, |A + B| = 0
A - B = {}, |A - B| = 0
B - A = {}, |B - A| = 0
A * B = {}, |A * B| = 0
A / B = {}, |A / B| = 0
B / A = {}, |B / A| = 0
проверка: |¬A| = |U| - |A|: 0 = 0 - 0 верно
проверка: |A ∪ B| + |A ∩ B| = |A| + |B|: 0 + 0 = 0 + 0 верно
```

Рисунок 24 — Результаты всех операций при $n = 0$

```
> m
      |·|
U      0
A      0
B      0
A∪B    0
A∩B    0
A\B    0
B\A    0
AΔB    0
¬A      0
¬B      0
A+B     0
A-B     0
B-A     0
A*B     0
A/B     0
B/A     0
```

Рисунок 25 — Матрица кратностей при $n = 0$

3.6 Сценарий 6: пересоздание универсума и конец ввода

Над универсумом $n = 2$, $k = 3$ ($|U| = 12$) случайно заполняются $A = \{00^1, 01^1, 10^2\}$ и $B = \{00^2, 10^1\}$ (рисунок 26). Затем создаётся универсум $n = 3$, $k = 2$, $|U| = 16$: программа выводит новый код и сообщает «А и В сброшены»: прежние A и B относились к старому носителю (рисунок 27). Мультимножества старого носителя не могут попасть в операции с новым универсумом: их признаки сняты, а при попытке смешать носители ядро выбросило бы `CarrierMismatchError`.

```
> a
|A| [0..12, Enter - случайно]: 4
r - случайно, m - вручную, Enter - отмена: r
A = {00×1, 01×1, 10×2}, |A| = 4

> b
|B| [0..12, Enter - случайно]: 3
r - случайно, m - вручную, Enter - отмена: r
B = {00×2, 10×1}, |B| = 3
```

Рисунок 26 — Заполнение A и B над универсумом с $n = 2$

```
> u
n [0..19]: 3
k [1..100, Enter - случайно]: 2
код Грея,  $2^3 = 8$ : 000 001 011 010 110 111 101 100
U = {000×2, 001×2, 011×2, 010×2, 110×2, 111×2, 101×2, 100×2}, |U| = 16
A и B сброшены
```

Рисунок 27 — Пересоздание универсума с $n = 3$

Начинается ручное заполнение A мощности 2, и на запросе строки поток ввода заканчивается (в консоли — `Ctrl+Z` и `Enter`, при вводе из файла — конец файла). Программа выводит «ввод закончился» и завершается с кодом 0, не заикливаясь на повторных запросах (рисунок 28).

```
> a
|A| [0..16, Enter - случайно]: 2
r - случайно, m - вручную, Enter - отмена: m
пары код:кратность через пробел (01:2 11), каждого кода не больше 2, Enter - отмена
A = {}, осталось 2:
ввод закончился
```

Рисунок 28 — Конец ввода во время ручного заполнения

3.7 Сценарий 7: наибольшая разрядность $n = 19$

Создаётся универсум $n = 19, k = 100: 2^{19} = 524\,288$ слов, $|U| = 52\,428\,800$ — наибольший возможный. Выводятся первые 16 слов кода (по пять в строке) с пометкой «и ещё 524272»; запись U обрезана после четырёх элементов с указанием числа скрытых, чтобы строка помещалась на экран (рисунок 29). A заполняется случайно со случайной мощностью $|A| = 4\,000\,752$; запись A обрезана так же (рисунок 30). Таблица команды $\mathfrak{t}$ показывает первые 32 строки; числовые столбцы расширены по длине $|U|$, строка « Σ » содержит полные мощности (рисунок 31). Весь сценарий, включая заполнение и вывод, выполняется за доли секунды, что согласуется с таблицей 5.

```
> u
n [0..19]: 19
k [1..100, Enter - случайно]: 100
код Грея,  $2^{19} = 524288$ :
0000000000000000 0000000000000001 0000000000000011 0000000000000010 00000000000000110
00000000000000111 00000000000000101 00000000000000100 000000000000001100 000000000000001101
000000000000001111 000000000000001110 000000000000001010 000000000000001011 000000000000001001
000000000000001000
... и ещё 524272
U = {0000000000000000×100, 0000000000000001×100, 0000000000000011×100, 0000000000000010×100, ... ещё 524284 эл.}, |U|
= 52428800
```

Рисунок 29 — Универсум при $n = 19, k = 100$

```
> a
|A| [0..52428800, Enter - случайно]:
|A| = 4000752
r - случайно, m - вручную, Enter - отмена: r
A = {0000000000000000×9, 0000000000000001×8, 0000000000000011×8, 0000000000000010×13, ... ещё 524025 эл.}, |A| = 40007
52
```

Рисунок 30 — Случайное заполнение A при $n = 19$

```
> t
```

код	kU	kA	kB
000000000000000000	100	9	-
000000000000000001	100	8	-
000000000000000011	100	8	-
000000000000000010	100	13	-
000000000000000110	100	4	-
000000000000000111	100	4	-
00000000000000101	100	7	-
00000000000000100	100	6	-
00000000000001100	100	4	-
00000000000001101	100	10	-
00000000000001111	100	7	-
00000000000001110	100	14	-
00000000000001010	100	6	-
00000000000001011	100	6	-
00000000000001001	100	6	-
00000000000001000	100	9	-
00000000000011000	100	7	-
00000000000011001	100	10	-
00000000000011011	100	8	-
00000000000011010	100	6	-
00000000000011110	100	14	-
00000000000011111	100	8	-
00000000000011101	100	9	-
00000000000011100	100	8	-
00000000000010100	100	11	-
00000000000010101	100	8	-
00000000000010111	100	8	-
00000000000010110	100	11	-
00000000000010010	100	12	-
00000000000010011	100	4	-
00000000000010001	100	2	-
00000000000010000	100	5	-
... показаны 32 из 524288			
Σ	52428800	4000752	-

Рисунок 31 — Таблица кратностей при $n = 19$ (первые 32 строки)

ЗАКЛЮЧЕНИЕ

Разработана консольная программа на C++, которая строит бинарный код Грея разрядности от 0 до 19 по номеру инвертируемого разряда, задаёт на нём универсум-мультимножество с одинаковой кратностью элементов k (вводится или выбирается случайно), заполняет A и B вручную или автоматически с заданной или случайной мощностью от 0 до $|U|$ и выводит тринадцать результатов операций с автоматической проверкой мощностей, таблицу и матрицу кратностей. Работа проверена на семи воспроизводимых сценариях: результаты совпали с ручным расчётом (таблица 13), с ожидаемыми значениями в граничных случаях $A = U$, $B = \emptyset$, $n = 0$, $k = 1$, а все некорректные значения отвергнуты с сообщением и повторным запросом.

Достоинства и недостатки.

- Генерация кода Грея не использует рекурсию и не хранит код меньшего порядка: каждое слово получается из предыдущего инвертированием одного разряда, всего $O(2^n)$ действий. Недостаток: слова выдаются только подряд; получить слово по номеру без предыдущих шагов алгоритм не позволяет (это делает формула $i \oplus \lfloor i/2 \rfloor$).
- Мультимножество хранится функцией кратности по номерам носителя: операции выполняются за $O(2^n)$ без поиска, универсум — объект того же класса, а результат любой операции остаётся подмультимножеством U (утверждение подраздела 1.3). Цена — память $O(2^n)$ на каждое мультимножество, даже почти пустое.
- Поиск элемента по коду выполняется за $O(n)$ через обратную таблицу `positions` ценой ещё 2^n чисел памяти.
- Случайное заполнение даёт точно заданную мощность, заполненные элементы сразу выбывают из выбора, поэтому холостых выборок нет. Недостаток: время $O(2^n + |A|)$, при $|A|$ порядка $|U|$ оно определяется $|U| = k \cdot 2^n$ и ограничивает $n_{\max} = 19$.
- Универсум с одинаковыми кратностями задаётся одним числом при любом n ; ограничение — все элементы универсума равноправны, задать разные $k_U(x)$ нельзя. При больших n вывод обрезается, и весь результат на экране не увидеть.
- Интерфейс: минималистичный, однобуквенные команды без рамок и псевдографики, сообщения с конкретной причиной отказа, ручной ввод стро-

кой пар, которая применяется целиком или не применяется вовсе, и само-проверка мощностей после операций. Недостаток: буквы команд латинские, при русской раскладке команда не распознаётся (программа сообщает об этом и повторяет запрос).

- При $n = 0$ принято $U = \{\}$, тогда как формально множество слов длины 0 состоит из одного пустого слова; соглашение выбрано потому, что пустое слово нельзя ни показать, ни ввести.

Масштабирование. Функциональность расширяется незначительными изменениями кода, так как математическая часть не зависит от интерфейса, все операции выражены через кратности по номерам, а меню строит вывод и таблицы по общему списку операций.

- *Новая операция* (умножение мультимножества на число λ : $\min\{\lambda k_A(x), k_U(x)\}$; проверка включения $A \subseteq B$ или равенства $A = B$): один метод `Multiset` с одним циклом по кратностям (для проверок уже есть `is_submultiset_of`) строка в массиве `OPERATIONS` и вызов в `Menu::compute_operations`; справка, список операций и матрица подхватят её автоматически.
- *Больше двух мультимножеств*: поля `a`, `b` заменяются массивом мультимножеств с именами и признаками; `fill_multiset` уже получает мультимножество, признак и имя параметрами, поэтому меняются только буквы команд и вызовы в `compute_operations`.
- *Разные кратности элементов универсума*: класс `Multiset`, все операции и `random_submultiset` уже работают с произвольными $k_U(x)$; достаточно добавить к `uniform_universe` функцию, которая заполняет кратности из вектора (введённого вручную или случайного), и запрос способа задания в `create_universe`.
- *Другой носитель* (обычный двоичный код, код Грея с другим порядком, произвольные слова): меняется только конструктор `CodeSet`; операции работают с номерами элементов и носитель не используют.
- *Вывод в файл*: `print_multiset` уже пишет в произвольный `std::ostream`; достаточно передать поток в `show_table` и `show_operations` вместо `std::cout`.
- *Пакетный режим*: уже работает — `lab1 --seed N < файл` выполняет сценарий и печатает протокол; так получены все рисунки раздела 3.

- *Большие n* : узкое место — `random_submultiset`, которая делает $|A|$ шагов. При $|A| > |U|/2$ выгоднее строить дополнение: начать с $A = U$ и удалять $|U| - |A|$ случайных единиц тем же способом (список элементов с ненулевой кратностью). Тогда шагов не больше $|U|/2$. Меняются функция `random_submultiset` и константа `MAX_DEPTH`; при $n > 31$ нужна 64-разрядная сборка (`MAX_TYPE_DEPTH` вычисляется из размера `size_t` автоматически).
- *Большие кратности*: константа `MAX_MULTIPPLICITY`; при этом нужно повторить замер (время автозаполнения в худшем случае пропорционально $K_{\max}$) и сохранить $K_{\max}^2 < 2^{32}$, чтобы не переполнялось произведение кратностей в 32-разрядной сборке.

В ходе работы освоены построение кода Грея по номеру изменяемого разряда с доказательством корректности, представление мультимножества функцией кратности и вычисление операций над ним, случайное заполнение подмультимножества заданной мощности без холостых выборов, обоснование ограничений программы замером, защищённый построчный ввод и разделение программы на математическую часть и интерфейс.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

- 1 Новиков, Ф. А. Дискретная математика для программистов : учебник для вузов / Ф. А. Новиков. — 3-е изд. — Санкт-Петербург : Питер, 2009. — 384 с.
- 2 Востров, А. В. Дискретная математика : курс лекций / А. В. Востров ; Санкт-Петербургский политехнический университет Петра Великого. — Санкт-Петербург, 2026. — URL: https://tema.spbstu.ru/dismath_lect/ (дата обращения: 04.10.2026). — Текст : электронный.

ПРИЛОЖЕНИЕ А

Текст программы

Листинг 3 — Makefile

```
1 # сборка: make (или mingw32-make в Windows); make lab1, make bench - одна
  цель; make clean - удалить
2 CXX = g++
3 CXXFLAGS = -std=c++14 -O2 -Wall -Wextra -Isrc
4
5 ifeq ($(OS),Windows_NT)
6     EXE = .exe
7 else
8     EXE =
9 endif
10
11 CORE_SOURCES = src/core/GrayCode.cpp src/core/Random.cpp
  src/core/Multiset.cpp src/core/Universe.cpp
12 UI_SOURCES = src/main.cpp src/ui/Input.cpp src/ui/Menu.cpp
13 HEADERS = $(wildcard src/core/*.hpp src/ui/*.hpp)
14
15 .PHONY: all lab1 bench clean
16
17 all: lab1 bench
18
19 lab1: lab1$(EXE)
20
21 # замер времени, по которому выбрана MAX_DEPTH
22 bench: bench$(EXE)
23
24 lab1$(EXE): $(UI_SOURCES) $(CORE_SOURCES) $(HEADERS)
25     $(CXX) $(CXXFLAGS) $(UI_SOURCES) $(CORE_SOURCES) -o $@
26
27 bench$(EXE): tools/bench.cpp $(CORE_SOURCES) $(HEADERS)
28     $(CXX) $(CXXFLAGS) tools/bench.cpp $(CORE_SOURCES) -o $@
29
30 # del - если make вызывает команды через cmd, rm - если через sh
31 clean:
32     -del /Q lab1$(EXE) bench$(EXE) 2>nul || rm -f lab1$(EXE) bench$(EXE)
```

Листинг 4 — src/core/Random.hpp

```
1 #pragma once
2 #include <cstdint>
3
4 // зерно для генератора: смесь std::random_device и текущего времени
```

```

5 unsigned make_seed();
6
7 // задаёт начальное состояние генератора; одно зерно — одна и та же
  последовательность чисел
8 void seed_random(unsigned seed);
9
10 // равновероятное целое число от 0 до bound - 1; bound должно быть больше
   нуля
11 size_t random_below(size_t bound);

```

Листинг 5 — src/core/Random.cpp

```

1 #include "core/Random.hpp"
2 #include <chrono>
3 #include <random>
4
5
6 // вихрь Мерсенна: период 2^19937 - 1, значения 32-битные (у std::rand всего
   15 бит)
7 static std::mt19937& generator(){
8     static std::mt19937 gen;
9     return gen;
10 }
11
12 unsigned make_seed(){
13     // в MinGW std::random_device детерминирован и при каждом запуске выдаёт
14     // одну и ту же последовательность, поэтому к нему примешивается время
15     std::random_device device;
16     unsigned time = static_cast<unsigned>(
17         std::chrono::high_resolution_clock::now().time_since_epoch().count()
18     );
19     return device() ^ time;
20 }
21
22 void seed_random(unsigned seed){
23     generator().seed(seed);
24 }
25
26 size_t random_below(size_t bound){
27     std::uniform_int_distribution<size_t> distribution(0, bound - 1);
28     return distribution(generator());
29 }

```

Листинг 6 — src/core/GrayCode.hpp

```

1 #pragma once
2 #include <cstdint>
3 #include <stdexcept>
4 #include <string>
5 #include <vector>
6
7 // наибольшая разрядность
8 const size_t MAX_DEPTH = 19;
9
10 // предел типа size_t
11 const size_t MAX_TYPE_DEPTH = sizeof(size_t) * 8 - 1;
12
13
14 class GrayCodeError : public std::runtime_error{
15 public:
16     explicit GrayCodeError(const std::string&
17         message):std::runtime_error(message){}
18 };
19
20 class InvalidDepthError : public GrayCodeError{
21 public:
22     InvalidDepthError():GrayCodeError("2^n не помещается в size_t"){ }
23 };
24
25 class CodeIndexError : public GrayCodeError{
26 public:
27     CodeIndexError():GrayCodeError("номер кода вне последовательности"){ }
28 };
29
30 // Генерация кода Грея
31 std::vector<size_t> generate_gray_code(size_t depth); // при depth = 0
32 // двоичная запись кода длиной depth, старший разряд слева
33 // последовательность пуста
34 std::string code_to_string(size_t code, size_t depth);
35
36 // носитель множеств
37 class CodeSet{
38     size_t depth;
39     std::vector<size_t> codes; // codes[i] – i-й код Грея
40     std::vector<size_t> positions; // positions[c] – номер кода c в
41     // последовательности
42 public:
43     explicit CodeSet(size_t depth);
44
45     size_t get_depth() const; // разрядность
46     size_t size() const; // число элементов

```

```

46     std::string get_code(size_t i) const;
47     bool find(const std::string& code, size_t& index) const;
48 };

```

Листинг 7 — src/core/GrayCode.cpp

```

1  #include "core/GrayCode.hpp"
2
3
4  // Функция, определяющая, какой бит перевернуть
5  static size_t changed_bit(size_t i){
6      size_t q = 0;
7      size_t j = i;
8      while(j % 2 == 0){
9          j /= 2;
10         q++;
11     }
12     // число нулей в конце двоичной записи i — номер бита справа, считая с
        нуля
13     return q;
14 }
15
16 std::vector<size_t> generate_gray_code(size_t depth){
17     /*
18         Алгоритм начинает с 0 и на каждом шаге переворачивает
19         один бит, какой бит перевернуть - решает функция changed_bit()
20     */
21     if(depth > MAX_TYPE_DEPTH){
22         throw InvalidDepthError();
23     }
24     std::vector<size_t> codes;
25     // при n = 0 кодов нет
26     if(depth == 0){
27         return codes;
28     }
29     size_t count = size_t(1) << depth;
30     codes.reserve(count);
31     size_t code = 0;
32     codes.push_back(code);
33     for(size_t i = 1; i < count; i++){
34         size_t p = changed_bit(i);
35         code ^= size_t(1) << p;
36         codes.push_back(code);
37     }
38     return codes;
39 }
40
41 // перевод кода Грея в строку

```

```

42 std::string code_to_string(size_t code, size_t depth){
43     std::string res;
44     for(size_t bit = depth; bit-- > 0;){
45         res += ((code >> bit) & 1) ? '1' : '0';
46     }
47     return res;
48 }
49
50
51 CodeSet::CodeSet(size_t
depth):depth(depth),codes(generate_gray_code(depth)),positions(codes.size()){
52     for(size_t i = 0; i < codes.size(); i++){
53         positions[codes[i]] = i;
54     }
55 }
56
57 // получить длину слова
58 size_t CodeSet::get_depth() const{
59     return depth;
60 }
61
62 // получить размер вектора
63 size_t CodeSet::size() const{
64     return codes.size();
65 }
66
67 // получить код по индексу
68 std::string CodeSet::get_code(size_t i) const{
69     if(i >= codes.size()){
70         throw CodeIndexError();
71     }
72     return code_to_string(codes[i], depth);
73 }
74
75 // получить индекс по коду
76 bool CodeSet::find(const std::string& code, size_t& index) const{
77     if(codes.empty() || code.size() != depth){
78         return false;
79     }
80     size_t value = 0;
81     for(size_t i = 0; i < code.size(); i++){
82         if(code[i] != '0' && code[i] != '1') return false;
83         value = value * 2 + (code[i] - '0'); // превращает строку в число
84     }
85     // все  $2^n$  слов длины n встречаются в коде Грея, поэтому номер есть у
любого value
86     index = positions[value];
87     return true;
88 }

```

Листинг 8 — src/core/Multiset.hpp

```

1  #pragma once
2  #include "core/GrayCode.hpp"
3  #include <cstdint>
4  #include <memory>
5  #include <ostream>
6  #include <stdexcept>
7  #include <string>
8  #include <vector>
9
10
11 class MultisetError : public std::runtime_error{
12 public:
13     explicit MultisetError(const std::string&
14         message):std::runtime_error(message){}
15 };
16
17 class MultisetIndexError : public MultisetError{
18 public:
19     MultisetIndexError():MultisetError("номер элемента вне носителя"){ }
20 };
21
22 class CarrierMismatchError : public MultisetError{
23 public:
24     CarrierMismatchError():MultisetError("мультимножества построены над
25         разными носителями"){ }
26 };
27
28 class NotSubmultisetError : public MultisetError{
29 public:
30     NotSubmultisetError():MultisetError("кратность элемента больше, чем в
31         универсуме"){ }
32 };
33
34 class CardinalityOverflowError : public MultisetError{
35 public:
36     CardinalityOverflowError():MultisetError("мощность больше мощности
37         ограничивающего мультимножества"){ }
38 };
39
40 // мультимножество над носителем — функция кратности: counts[i] = k(x_i), x_i
41 // — i-й код Грея;
42 // универсум U и мультимножества A, B — объекты этого класса над общим
43 // носителем
44 class Multiset{
45     std::shared_ptr<const CodeSet> carrier; // общий носитель

```

```

41     std::vector<size_t> counts; // кратности
42
43     // Сравнивает, одинаковый ли указатель у носителей
44     void check_same_carrier(const Multiset& other) const; // -
        CarrierMismatchError
45     // доп проверка на подмножество
46     void check_within(const Multiset& universe) const; // -
        NotSubmultisetError
47
48 public:
49     explicit Multiset(std::shared_ptr<const CodeSet> carrier); // все
        кратности равны нулю
50
51     const CodeSet& get_carrier() const; // Ссылка на носитель
52     size_t size() const; // число элементов носителя
53     size_t get_count(size_t i) const; // получить кратность элемента
54     void set_count(size_t i, size_t count);
55     size_t cardinality() const; // сумма кратностей - мощность
56     bool is_submultiset_of(const Multiset& other) const;
57
58     // === операции ===
59
60     //теор. мн.
61     Multiset unite(const Multiset& b) const; // объединение
62     Multiset intersect(const Multiset& b) const; // пересечение
63     Multiset complement(const Multiset& universe) const; // дополнение
64     Multiset difference(const Multiset& b, const Multiset& universe) const;
        // разность
65     Multiset symmetric_difference(const Multiset& b, const Multiset&
        universe) const; // симметрическая разность
66
67     //арифм.
68     Multiset arithmetic_sum(const Multiset& b, const Multiset& universe)
        const; // сумма
69     Multiset arithmetic_difference(const Multiset& b) const; // разность
70     Multiset arithmetic_product(const Multiset& b, const Multiset& universe)
        const; // произведение
71     Multiset arithmetic_division(const Multiset& b) const; // деление
72
73     friend Multiset random_submultiset(const Multiset& bound, size_t
        cardinality); // автоматическое заполнение
74 };
75
76 // случайное подмультимножество bound мощности cardinality: cardinality раз
    случайный
77 // незаполненный элемент получает +1 к кратности
78 Multiset random_submultiset(const Multiset& bound, size_t cardinality);
79
80 // запись вида {000×2, 011×1}; элементы с нулевой кратностью не выводятся;

```

```

81 void print_multiset(std::ostream& out, const Multiset& multiset, size_t
    limit);
82 std::ostream& operator<<(std::ostream& out, const Multiset& multiset);

```

Листинг 9 — src/core/Multiset.cpp

```

1  #include "core/Multiset.hpp"
2  #include "core/Random.hpp"
3  #include <algorithm>
4
5
6  Multiset::Multiset(std::shared_ptr<const CodeSet>
    carrier):carrier(carrier),counts(carrier->size(), 0){}
7
8  void Multiset::check_same_carrier(const Multiset& other) const{
9      if(carrier != other.carrier){
10         throw CarrierMismatchError();
11     }
12 }
13
14 void Multiset::check_within(const Multiset& universe) const{
15     // носители сравнивает сам is_submultiset_of
16     if(!is_submultiset_of(universe)){
17         throw NotSubmultisetError();
18     }
19 }
20
21 const CodeSet& Multiset::get_carrier() const{
22     return *carrier;
23 }
24
25 size_t Multiset::size() const{
26     return counts.size();
27 }
28
29 size_t Multiset::get_count(size_t i) const{
30     if(i >= counts.size()){
31         throw MultisetIndexError();
32     }
33     return counts[i];
34 }
35
36 void Multiset::set_count(size_t i, size_t count){
37     if(i >= counts.size()){
38         throw MultisetIndexError();
39     }
40     counts[i] = count;

```

```

41 }
42
43 size_t Multiset::cardinality() const{
44     size_t res = 0;
45     for(size_t i = 0; i < counts.size(); i++){
46         res += counts[i];
47     }
48     return res;
49 }
50
51 bool Multiset::is_submultiset_of(const Multiset& other) const{
52     check_same_carrier(other);
53     for(size_t i = 0; i < counts.size(); i++){
54         if(counts[i] > other.counts[i]) return false;
55     }
56     return true;
57 }
58
59 Multiset Multiset::unite(const Multiset& b) const{
60     check_same_carrier(b);
61     Multiset res(carrier);
62     for(size_t i = 0; i < counts.size(); i++){
63         res.counts[i] = std::max(counts[i], b.counts[i]);
64     }
65     return res;
66 }
67
68 Multiset Multiset::intersect(const Multiset& b) const{
69     check_same_carrier(b);
70     Multiset res(carrier);
71     for(size_t i = 0; i < counts.size(); i++){
72         res.counts[i] = std::min(counts[i], b.counts[i]);
73     }
74     return res;
75 }
76
77 Multiset Multiset::complement(const Multiset& universe) const{
78     check_within(universe);
79     Multiset res(carrier);
80     for(size_t i = 0; i < counts.size(); i++){
81         res.counts[i] = universe.counts[i] - counts[i]; // гарантированно >=
            0, т.к. A подмножество U
82     }
83     return res;
84 }
85
86 Multiset Multiset::difference(const Multiset& b, const Multiset& universe)
87 const{
    //  $A \setminus B = A \cap \neg B$ 

```

```

88     return intersect(b.complement(universe));
89 }
90
91 Multiset Multiset::symmetric_difference(const Multiset& b, const Multiset&
universe) const{
92     //  $A \triangle B = (A \setminus B) \cup (B \setminus A)$ 
93     return difference(b, universe).unite(b.difference(*this, universe));
94 }
95
96 Multiset Multiset::arithmetic_sum(const Multiset& b, const Multiset&
universe) const{
97     check_within(universe);
98     b.check_within(universe);
99     Multiset res(carrier);
100    for(size_t i = 0; i < counts.size(); i++){
101        res.counts[i] = std::min(counts[i] + b.counts[i],
universe.counts[i]);
102    }
103    return res;
104 }
105
106 Multiset Multiset::arithmetic_difference(const Multiset& b) const{
107     check_same_carrier(b);
108     Multiset res(carrier);
109     for(size_t i = 0; i < counts.size(); i++){
110        res.counts[i] = (counts[i] > b.counts[i]) ? counts[i] - b.counts[i] :
0;
111    }
112    return res;
113 }
114
115 Multiset Multiset::arithmetic_product(const Multiset& b, const Multiset&
universe) const{
116     check_within(universe);
117     b.check_within(universe);
118     Multiset res(carrier);
119     for(size_t i = 0; i < counts.size(); i++){
120        res.counts[i] = std::min(counts[i] * b.counts[i],
universe.counts[i]);
121    }
122    return res;
123 }
124
125 Multiset Multiset::arithmetic_division(const Multiset& b) const{
126     check_same_carrier(b);
127     Multiset res(carrier);
128     for(size_t i = 0; i < counts.size(); i++){
129        res.counts[i] = (b.counts[i] != 0) ? counts[i] / b.counts[i] : 0;
130    }

```

```

131     return res;
132 }
133
134
135 Multiset random_submultiset(const Multiset& bound, size_t cardinality){
136     if(cardinality > bound.cardinality()){
137         throw CardinalityOverflowError();
138     }
139     Multiset res(bound.carrier);
140     // номера элементов, кратность которых в res ещё меньше, чем в bound
141     std::vector<size_t> free;
142     free.reserve(bound.counts.size());
143     for(size_t i = 0; i < bound.counts.size(); i++){
144         if(bound.counts[i] > 0) free.push_back(i);
145     }
146     // cardinality раз: случайный элемент со свободным местом получает +1 к
    кратности;
147     // список не пустеет раньше времени, так как свободных мест |bound| -
    step > 0
148     for(size_t step = 0; step < cardinality; step++){
149         size_t j = random_below(free.size());
150         size_t i = free[j];
151         res.counts[i]++;
152         if(res.counts[i] == bound.counts[i]){
153             // элемент заполнен: на его место в списке встаёт последний,
            список короче на 1
154             free[j] = free.back();
155             free.pop_back();
156         }
157     }
158     return res;
159 }
160
161
162 void print_multiset(std::ostream& out, const Multiset& multiset, size_t
    limit){
163     const CodeSet& carrier = multiset.get_carrier();
164     size_t shown = 0;
165     size_t hidden = 0;
166     out << "{";
167     for(size_t i = 0; i < multiset.size(); i++){
168         size_t count = multiset.get_count(i);
169         if(count == 0) continue;
170         if(shown == limit){
171             hidden++;
172             continue;
173         }
174         if(shown > 0) out << ", ";
175         out << carrier.get_code(i) << "x" << count;

```

```

176         shown++;
177     }
178     if(hidden > 0) out << ", ... ещё " << hidden << " эл.";
179     out << "}";
180 }
181
182 std::ostream& operator<<(std::ostream& out, const Multiset& multiset){
183     print_multiset(out, multiset, multiset.size());
184     return out;
185 }

```

Листинг 10 — src/core/Universe.hpp

```

1  #pragma once
2  #include "core/GrayCode.hpp"
3  #include "core/Multiset.hpp"
4  #include <cstdint>
5  #include <memory>
6
7  // наибольшая кратность элемента универсума
8  const size_t MAX_MULTIPPLICITY = 100;
9
10
11 class InvalidMultiplicityError : public MultisetError{
12 public:
13     InvalidMultiplicityError():MultisetError("кратность элемента универсума
14     вне диапазона от 1 до MAX_MULTIPPLICITY"){ }
15 };
16
17 // универсум, в котором каждый код Грея входит с одной и той же кратностью
18 // multiplicity
19 Multiset uniform_universe(std::shared_ptr<const CodeSet> carrier, size_t
20 multiplicity);

```

Листинг 11 — src/core/Universe.cpp

```

1  #include "core/Universe.hpp"
2
3
4  Multiset uniform_universe(std::shared_ptr<const CodeSet> carrier, size_t
5  multiplicity){
6      if(multiplicity < 1 || multiplicity > MAX_MULTIPPLICITY){
7          throw InvalidMultiplicityError();
8      }
9      Multiset universe(carrier);

```

```

9     for(size_t i = 0; i < universe.size(); i++){
10         universe.set_count(i, multiplicity);
11     }
12     return universe;
13 }

```

Листинг 12 — src/ui/Input.hpp

```

1  #pragma once
2  #include <cstdint>
3  #include <stdexcept>
4  #include <string>
5
6  // ввод закончился (Ctrl+Z в Windows, Ctrl+D в Linux, конец файла)
7  class InputClosedError : public std::runtime_error{
8  public:
9      InputClosedError():std::runtime_error("ввод закончился"){}
10 };
11
12 // печатает приглашение и читает строку без пробелов, табуляций и \r по краям
13 std::string read_text(const std::string& prompt);
14
15 // строка из 1..9 цифр; число записывается в value
16 bool parse_number(const std::string& text, size_t& value);
17
18 // строка длины depth из символов 0 и 1
19 bool is_code(const std::string& text, size_t depth);
20
21 // переспрашивает, пока не будет введено целое число от min до max
22 size_t read_number(const std::string& prompt, size_t min, size_t max);
23
24 // то же, но пустая строка допустима: тогда возвращается false, value не
    меняется
25 bool read_number_or_empty(const std::string& prompt, size_t min, size_t max,
    size_t& value);
26
27 // переспрашивает, пока не будет введена одна буква из letters (регистр не
    важен);
28 // если allow_empty, пустая строка допустима и возвращается '\0'
29 char read_letter(const std::string& prompt, const std::string& letters, bool
    allow_empty);

```

Листинг 13 — src/ui/Input.cpp

```

1  #include "ui/Input.hpp"
2  #include <iostream>

```

```

3 #ifdef _WIN32
4 #include <io.h>
5 static bool stdin_is_console(){ return _isatty(0) != 0; }
6 #else
7 #include <unistd.h>
8 static bool stdin_is_console(){ return isatty(0) != 0; }
9 #endif
10
11
12 std::string read_text(const std::string& prompt){
13     std::cout << prompt;
14     std::string line;
15     if(!std::getline(std::cin, line)){
16         throw InputClosedError();
17     }
18     // при вводе из файла строка не отображается консолью, поэтому программа
19     // выводит её сама
20     static const bool echo = !stdin_is_console();
21     if(echo) std::cout << line.substr(0, line.find('\r')) << std::endl;
22     size_t begin = line.find_first_not_of(" \t\r");
23     if(begin == std::string::npos) return "";
24     size_t end = line.find_last_not_of(" \t\r");
25     return line.substr(begin, end - begin + 1);
26 }
27
28 // не больше 9 цифр: число до 999 999 999 гарантированно помещается в size_t
29 bool parse_number(const std::string& text, size_t& value){
30     if(text.empty() || text.size() > 9) return false;
31     size_t res = 0;
32     for(size_t i = 0; i < text.size(); i++){
33         if(text[i] < '0' || text[i] > '9') return false;
34         res = res * 10 + (text[i] - '0');
35     }
36     value = res;
37     return true;
38 }
39
40 bool is_code(const std::string& text, size_t depth){
41     if(text.size() != depth) return false;
42     for(size_t i = 0; i < text.size(); i++){
43         if(text[i] != '0' && text[i] != '1') return false;
44     }
45     return true;
46 }
47
48 // сообщение называет причину: пустая строка, не число или число вне отрезка
49 static void report_bad_number(const std::string& text, size_t min, size_t
max){

```

```

49     std::string range = "[" + std::to_string(min) + ".." +
std::to_string(max) + "]";
50     size_t value = 0;
51     if(text.empty()){
52         std::cout << "! пустая строка: нужно целое число из " << range <<
std::endl;
53     }
54     else if(parse_number(text, value)){
55         std::cout << "! " << value << " не входит в " << range << std::endl;
56     }
57     else{
58         std::cout << "! «" << text << "» не целое неотрицательное число;
нужно число из " << range << std::endl;
59     }
60 }
61
62 size_t read_number(const std::string& prompt, size_t min, size_t max){
63     while(true){
64         std::string line = read_text(prompt);
65         size_t value = 0;
66         if(parse_number(line, value) && value >= min && value <= max){
67             return value;
68         }
69         report_bad_number(line, min, max);
70     }
71 }
72
73 bool read_number_or_empty(const std::string& prompt, size_t min, size_t max,
size_t& value){
74     while(true){
75         std::string line = read_text(prompt);
76         if(line.empty()) return false;
77         size_t res = 0;
78         if(parse_number(line, res) && res >= min && res <= max){
79             value = res;
80             return true;
81         }
82         report_bad_number(line, min, max);
83     }
84 }
85
86 char read_letter(const std::string& prompt, const std::string& letters, bool
allow_empty){
87     std::string listed;
88     for(size_t i = 0; i < letters.size(); i++){
89         listed += (i > 0 ? " " : "") + std::string(1, letters[i]);
90     }
91     while(true){
92         std::string line = read_text(prompt);

```

```

93     if(line.empty() && allow_empty) return '\0';
94     if(line.size() == 1){
95         char letter = line[0];
96         if(letter >= 'A' && letter <= 'Z') letter = letter - 'A' + 'a';
97         if(letters.find(letter) != std::string::npos) return letter;
98     }
99     if(line.empty()){
100         std::cout << "! пустая строка: введите одну из букв " << listed
101         << std::endl;
102     }
103     else{
104         std::cout << "! «" << line << "» не команда; допустимы буквы " <<
105         listed << std::endl;
106     }
107 }

```

Листинг 14 — src/ui/Menu.hpp

```

1  #pragma once
2  #include "core/GrayCode.hpp"
3  #include "core/Multiset.hpp"
4  #include <cstdlib>
5  #include <memory>
6  #include <string>
7  #include <vector>
8
9  class Menu{
10     std::shared_ptr<const CodeSet> carrier;    // общий носитель U, A и B
11     Multiset universe;
12     Multiset a;
13     Multiset b;
14     bool universe_ready;
15     bool a_ready;
16     bool b_ready;
17
18     bool is_available(char command) const;
19
20     void create_universe();
21     void fill_multiset(Multiset& multiset, bool& ready, const std::string&
22     name);
23     bool fill_manual(Multiset& target, size_t cardinality, const std::string&
24     name) const;
25     bool parse_manual_line(const std::string& line, const Multiset& target,
26     size_t left,
27
28     std::vector<size_t>& indices, std::vector<size_t>&
29     counts) const;

```

```

25
26     std::vector<Multiset> compute_operations() const;
27     void show_help() const;
28     void show_table() const;
29     void show_operations() const;
30     void show_matrix() const;
31
32 public:
33     Menu();
34     void run();
35 };

```

Листинг 15 — src/ui/Menu.cpp

```

1  #include "ui/Menu.hpp"
2  #include "core/Random.hpp"
3  #include "core/Universe.hpp"
4  #include "ui/Input.hpp"
5  #include <algorithm>
6  #include <iostream>
7  #include <sstream>
8
9
10 static const size_t TABLE_LIMIT = 32;    // наибольшее число строк таблицы
11 static const size_t PRINT_LIMIT = 16;     // наибольшее число элементов
    мультимножества и слов кода в выводе
12 static const size_t LINE_WIDTH = 100;    // примерная наибольшая ширина
    записи мультимножества при n > 4
13 static const size_t MATRIX_WIDTH = 112;  // наибольшая ширина строки матрицы
    кратностей
14
15 // операции в порядке вывода; результаты compute_operations идут в том же
    порядке
16 struct OperationInfo{
17     const char* title;    // название
18     const char* notation; // обозначение
19     const char* formula;  // кратность элемента в результате
20 };
21
22 static const OperationInfo OPERATIONS[] = {
23     {"объединение", "A ∪ B", "max(kA, kB)"},
24     {"пересечение", "A ∩ B", "min(kA, kB)"},
25     {"разность", "A \\ B", "min(kA, kU - kB)"},
26     {"разность", "B \\ A", "min(kB, kU - kA)"},
27     {"симметрическая разность", "A △ B", "max(k[A\\B], k[B\\A])"},
28     {"дополнение", "¬A", "kU - kA"},
29     {"дополнение", "¬B", "kU - kB"},

```

```

30     {"арифметическая сумма", "A + B", "min(kA + kB, kU)"},
31     {"арифметическая разность", "A - B", "max(kA - kB, 0)"},
32     {"арифметическая разность", "B - A", "max(kB - kA, 0)"},
33     {"арифметическое произведение", "A * B", "min(kA * kB, kU)"},
34     {"арифметическое деление", "A / B", "kA div kB; 0 при kB = 0"},
35     {"арифметическое деление", "B / A", "kB div kA; 0 при kA = 0"},
36 };
37 static const size_t OPERATION_COUNT = sizeof(OPERATIONS) /
    sizeof(OPERATIONS[0]);
38
39 static const char* COMMANDS = "u - универсум, a/b - задать A/B, t - таблица,
    o - операции, "
40
41                                     "m - матрица, h - справка, q - выход";
42
43 // ширина строки в символах: в UTF-8 продолжения символа имеют вид 10xxxxxx
44 static size_t text_width(const std::string& text){
45     size_t width = 0;
46     for(size_t i = 0; i < text.size(); i++){
47         if((static_cast<unsigned char>(text[i]) & 0xC0) != 0x80) width++;
48     }
49     return width;
50 }
51
52 static std::string pad_right(const std::string& text, size_t width){
53     size_t current = text_width(text);
54     return current >= width ? text : text + std::string(width - current, '
    ');
55 }
56
57 static std::string pad_left(const std::string& text, size_t width){
58     size_t current = text_width(text);
59     return current >= width ? text : std::string(width - current, ' ') +
    text;
60 }
61
62 // при n <= 4 запись полная (не больше 2^4 = PRINT_LIMIT элементов); при
    больших n элементов
63 // столько, сколько помещается примерно в LINE_WIDTH символов, но не меньше 4
64 static std::string format_multiset(const Multiset& multiset){
65     size_t depth = multiset.get_carrier().get_depth();
66     size_t limit = PRINT_LIMIT;
67     if(depth > 4){
68         size_t element_width = depth + 6;    // "код×100, "
69         limit = std::min(PRINT_LIMIT, std::max<size_t>(4, LINE_WIDTH /
    element_width));
70     }
71     std::ostringstream out;
72     print_multiset(out, multiset, limit);

```

```

73     return out.str();
74 }
75
76 // обозначение без пробелов для заголовков столбцов: A U B -> AUB
77 static std::string compact(const std::string& notation){
78     std::string res = notation;
79     res.erase(std::remove(res.begin(), res.end(), ' '), res.end());
80     return res;
81 }
82
83
84 Menu::Menu():carrier(std::make_shared<CodeSet>(0)),universe(carrier),a(carri
er),b(carrier),
85     universe_ready(false),a_ready(false),b_ready(false){}
86
87 void Menu::run(){
88     std::cout << COMMANDS << std::endl;
89     while(true){
90         std::cout << std::endl;
91         char command = read_letter("> ", "uabtomhq", false);
92         if(command == 'q') return;
93         if(!is_available(command)) continue;
94         switch(command){
95             case 'u': create_universe(); break;
96             case 'a': fill_multiset(a, a_ready, "A"); break;
97             case 'b': fill_multiset(b, b_ready, "B"); break;
98             case 't': show_table(); break;
99             case 'o': show_operations(); break;
100            case 'm': show_matrix(); break;
101            case 'h': show_help(); break;
102        }
103    }
104 }
105
106 // печатает, чего не хватает для команды, и возвращает false, если команду
    ВЫПОЛНИТЬ НЕЛЬЗЯ
107 bool Menu::is_available(char command) const{
108     if(command == 'u' || command == 'h') return true;
109     if(!universe_ready){
110         std::cout << "! команда " << command << " недоступна: универсум ещё
            не создан (команда u)" << std::endl;
111         return false;
112     }
113     if((command == 'o' || command == 'm') && (!a_ready || !b_ready)){
114         std::string missing;
115         if(!a_ready) missing = "A (команда a)";
116         if(!b_ready) missing += std::string(missing.empty() ? "" : " и ") +
            "B (команда b)";

```

```

117         std::cout << "! команда " << command << " недоступна: не задано " <<
        missing << std::endl;
118         return false;
119     }
120     return true;
121 }
122
123 void Menu::create_universe(){
124     size_t depth = read_number("n [0.." + std::to_string(MAX_DEPTH) + "]: ",
        0, MAX_DEPTH);
125     std::shared_ptr<const CodeSet> new_carrier =
        std::make_shared<CodeSet>(depth);
126     size_t size = new_carrier->size();
127     Multiset new_universe(new_carrier);
128     size_t multiplicity = 0;
129
130     if(size == 0){
131         // элементов нет, поэтому и кратность задавать некому
132         std::cout << "при n = 0 в коде Грея нет слов, кратность не нужна" <<
            std::endl;
133     }
134     else{
135         if(!read_number_or_empty("k [1.." + std::to_string(MAX_MULTIPLICITY)
            + ", Enter - случайно]: ",
136             1, MAX_MULTIPLICITY, multiplicity)){
137             multiplicity = 1 + random_below(MAX_MULTIPLICITY);
138             std::cout << "k = " << multiplicity << std::endl;
139         }
140         new_universe = uniform_universe(new_carrier, multiplicity);
141     }
142
143     bool had_multisets = a_ready || b_ready;
144     carrier = new_carrier;
145     universe = new_universe;
146     a = Multiset(carrier);
147     b = Multiset(carrier);
148     universe_ready = true;
149     a_ready = false;
150     b_ready = false;
151
152     // не больше PRINT_LIMIT слов кода, в строке столько, сколько помещается
        в LINE_WIDTH
153     if(size > 0){
154         size_t shown = std::min(size, PRINT_LIMIT);
155         size_t per_line = std::max<size_t>(1, LINE_WIDTH / (depth + 1));
156         std::cout << "код Грея, 2^" << depth << " = " << size << ":";
157         for(size_t i = 0; i < shown; i++){
158             if(shown > per_line && i % per_line == 0) std::cout << std::endl;
159             else if(i > 0 || shown <= per_line) std::cout << " ";

```

```

160         std::cout << carrier->get_code(i);
161     }
162     std::cout << std::endl;
163     if(shown < size) std::cout << "... и ещё " << size - shown <<
        std::endl;
164 }
165 std::cout << "U = " << format_multiset(universe) << ", |U| = " <<
    universe.cardinality() << std::endl;
166 if(had_multisets) std::cout << "A и B сброшены" << std::endl;
167 }
168
169 void Menu::fill_multiset(Multiset& multiset, bool& ready, const std::string&
    name) {
170     size_t high = universe.cardinality();
171     size_t cardinality = 0;
172     if(!read_number_or_empty("|" + name + "| [0.." + std::to_string(high) +
        ", Enter - случайно]: ",
173         0, high, cardinality)){
174         cardinality = random_below(high + 1);
175         std::cout << "|" << name << "| = " << cardinality << std::endl;
176     }
177     char method = read_letter("r - случайно, m - вручную, Enter - отмена: ",
        "rm", true);
178     if(method == '\0'){
179         std::cout << "отменено" << std::endl;
180         return;
181     }
182
183     // заполняется копия: при отмене ручного ввода прежнее мультимножество
        сохраняется
184     Multiset filled(carrier);
185     if(method == 'm'){
186         if(!fill_manual(filled, cardinality, name)){
187             std::cout << "отменено" << std::endl;
188             return;
189         }
190     }
191     else{
192         filled = random_submultiset(universe, cardinality);
193     }
194
195     multiset = filled;
196     ready = true;
197     std::cout << name << " = " << format_multiset(multiset) << ", |" << name
        << "| = "
198         << multiset.cardinality() << std::endl;
199 }
200

```

```

201 bool Menu::fill_manual(Multiset& target, size_t cardinality, const
std::string& name) const{
202     if(cardinality == 0) return true;
203     std::cout << "пары код:кратность через пробел (01:2 11), каждого кода не
        больше "
204         << universe.get_count(0) << ", Enter - отмена" << std::endl;
205     while(target.cardinality() < cardinality){
206         size_t left = cardinality - target.cardinality();
207         std::string line = read_text(name + " = " + format_multiset(target) +
        ", осталось " + std::to_string(left) + ": ");
208         if(line.empty()) return false;
209         std::vector<size_t> indices;
210         std::vector<size_t> counts;
211         // строка применяется только целиком: при ошибке в любой паре A не
        меняется
212         if(parse_manual_line(line, target, left, indices, counts)){
213             for(size_t i = 0; i < indices.size(); i++){
214                 target.set_count(indices[i], target.get_count(indices[i]) +
        counts[i]);
215             }
216         }
217     }
218     return true;
219 }
220
221 // разбирает строку пар код:кратность; при ошибке печатает её причину и
        возвращает false
222 bool Menu::parse_manual_line(const std::string& line, const Multiset& target,
        size_t left,
223         std::vector<size_t>& indices,
        std::vector<size_t>& counts) const{
224     size_t depth = carrier->get_depth();
225     size_t total = 0;
226     std::istringstream words(line);
227     std::string word;
228     while(words >> word){
229         size_t colon = word.find(':');
230         std::string code = word.substr(0, colon);
231         size_t count = 1;
232         if(!is_code(code, depth)){
233             std::cout << "! «" << word << "»: код должен состоять из " <<
        depth << " символов 0 и 1" << std::endl;
234             return false;
235         }
236         if(colon != std::string::npos && (!parse_number(word.substr(colon +
        1), count) || count == 0)){
237             std::cout << "! «" << word << "»: после двоеточия нужна кратность
        - целое число от 1" << std::endl;
238             return false;

```

```

239     }
240     size_t index = 0;
241     carrier->find(code, index);    // код длины n из 0 и 1 всегда есть в
                                     носителе
242     // учитываются и единицы этого же кода, уже записанные в строке
                                     раньше
243     size_t taken = target.get_count(index);
244     for(size_t i = 0; i < indices.size(); i++){
245         if(indices[i] == index) taken += counts[i];
246     }
247     size_t free = universe.get_count(index) - std::min(taken,
                                     universe.get_count(index));
248     if(count > free){
249         std::cout << "! «" << word << "»: у " << code << " свободно " <<
                                     free << " (kU = "
250                                     << universe.get_count(index) << ", занято " << taken <<
                                     ")" << std::endl;
251         return false;
252     }
253     indices.push_back(index);
254     counts.push_back(count);
255     total += count;
256 }
257 if(total > left){
258     std::cout << "! в строке " << total << " ед., а осталось распределить
                                     " << left << std::endl;
259     return false;
260 }
261 return true;
262 }
263
264 std::vector<Multiset> Menu::compute_operations() const{
265     std::vector<Multiset> res;
266     res.push_back(a.unite(b));
267     res.push_back(a.intersect(b));
268     res.push_back(a.difference(b, universe));
269     res.push_back(b.difference(a, universe));
270     res.push_back(a.symmetric_difference(b, universe));
271     res.push_back(a.complement(universe));
272     res.push_back(b.complement(universe));
273     res.push_back(a.arithmetic_sum(b, universe));
274     res.push_back(a.arithmetic_difference(b));
275     res.push_back(b.arithmetic_difference(a));
276     res.push_back(a.arithmetic_product(b, universe));
277     res.push_back(a.arithmetic_division(b));
278     res.push_back(b.arithmetic_division(a));
279     return res;
280 }
281

```

```

282 void Menu::show_help() const{
283     std::cout << COMMANDS << std::endl
284         << "запись {000×2, 011×1}: код Грея × кратность, {} - пустое;
           kU, kA, kB - кратности в U, A, B" << std::endl;
285     for(size_t i = 0; i < OPERATION_COUNT; i++){
286         std::cout << pad_right(OPERATIONS[i].notation, 7) <<
           pad_right(OPERATIONS[i].formula, 26)
287             << OPERATIONS[i].title << std::endl;
288     }
289     std::cout << "0 ≤ n ≤ " << MAX_DEPTH << ", 1 ≤ k ≤ " <<
           MAX_MULTIPLICITY << ", 0 ≤ |A|, |B| ≤ |U|" << std::endl;
290 }
291
292 void Menu::show_table() const{
293     size_t code_width = std::max<size_t>(carrier->get_depth(), 3);
294     // числа выравниваются по самому длинному из них - мощности |U| в строке
           Σ
295     size_t number_width =
           std::max<size_t>(std::to_string(universe.cardinality()).size(), 3) + 1;
296     std::cout << pad_right("код", code_width) << pad_left("kU", number_width)
297         << pad_left("kA", number_width) << pad_left("kB", number_width)
           << std::endl;
298     size_t rows = std::min(universe.size(), TABLE_LIMIT);
299     for(size_t i = 0; i < rows; i++){
300         std::cout << pad_right(carrier->get_code(i), code_width)
301             << pad_left(std::to_string(universe.get_count(i)),
           number_width)
302             << pad_left(a_ready ? std::to_string(a.get_count(i)) : "-",
           number_width)
303             << pad_left(b_ready ? std::to_string(b.get_count(i)) : "-",
           number_width) << std::endl;
304     }
305     if(rows < universe.size()){
306         std::cout << "... показаны " << rows << " из " << universe.size() <<
           std::endl;
307     }
308     std::cout << pad_right("Σ", code_width)
309         << pad_left(std::to_string(universe.cardinality()),
           number_width)
310         << pad_left(a_ready ? std::to_string(a.cardinality()) : "-",
           number_width)
311         << pad_left(b_ready ? std::to_string(b.cardinality()) : "-",
           number_width) << std::endl;
312 }
313
314 void Menu::show_operations() const{
315     std::vector<Multiset> results = compute_operations();
316     for(size_t i = 0; i < OPERATION_COUNT; i++){

```

```

317         std::cout << pad_right(OPERATIONS[i].notation, 6) << "= " <<
            format_multiset(results[i])
318         << ", |" << OPERATIONS[i].notation << "| = " <<
            results[i].cardinality() << std::endl;
319     }
320
321     // следствия определений:  $|\neg A| = |U| - |A|$  и  $|A \cup B| + |A \cap B| = |A| + |B|$ 
322     size_t u = universe.cardinality();
323     size_t na = a.cardinality();
324     size_t nb = b.cardinality();
325     size_t complement = results[5].cardinality();
326     size_t unite = results[0].cardinality();
327     size_t intersect = results[1].cardinality();
328     std::cout << "проверка:  $|\neg A| = |U| - |A|$ : " << complement << " = " << u
        << " - " << na
329         << (complement == u - na ? " верно" : " НЕВЕРНО") << std::endl;
330     std::cout << "проверка:  $|A \cup B| + |A \cap B| = |A| + |B|$ : " << unite << " +
        " << intersect << " = " << na << " + " << nb
331         << (unite + intersect == na + nb ? " верно" : " НЕВЕРНО") <<
            std::endl;
332 }
333
334 void Menu::show_matrix() const{
335     std::vector<Multiset> results = compute_operations();
336     std::vector<std::string> titles = {"U", "A", "B"};
337     std::vector<const Multiset*> rows = {&universe, &a, &b};
338     for(size_t i = 0; i < OPERATION_COUNT; i++){
339         titles.push_back(compact(OPERATIONS[i].notation));
340         rows.push_back(&results[i]);
341     }
342
343     // столбцов столько, сколько помещается в MATRIX_WIDTH
344     const size_t label_width = 4;
345     const size_t total_width = 9;
346     size_t column_width = std::max<size_t>(carrier->get_depth(), 3) + 2;
347     size_t columns = std::min(universe.size(), (MATRIX_WIDTH - label_width -
        total_width) / column_width);
348
349     std::cout << pad_right("", label_width);
350     for(size_t j = 0; j < columns; j++) std::cout <<
        pad_left(carrier->get_code(j), column_width);
351     std::cout << pad_left("|·|", total_width) << std::endl;
352     for(size_t i = 0; i < rows.size(); i++){
353         std::cout << pad_right(titles[i], label_width);
354         for(size_t j = 0; j < columns; j++){
355             std::cout << pad_left(std::to_string(rows[i]->get_count(j)),
                column_width);
356         }

```

```

357         std::cout << pad_left(std::to_string(rows[i]->cardinality()),
358                                total_width) << std::endl;
359     }
360     if(columns < universe.size()){
361         std::cout << "... показаны " << columns << " из " << universe.size()
362         << " кодов, мощности по всем" << std::endl;
363     }
364 }

```

Листинг 16 — src/main.cpp

```

1  #include "core/Random.hpp"
2  #include "ui/Input.hpp"
3  #include "ui/Menu.hpp"
4  #include <cstdlib>
5  #include <cstring>
6  #include <exception>
7  #include <iostream>
8  #ifdef _WIN32
9  #include <windows.h>
10 #endif
11
12
13 int main(int argc, char* argv[]){
14     #ifdef _WIN32
15         // строки программы записаны в UTF-8, а консоль Windows по умолчанию
16         // выводит в кодировке 866
17         SetConsoleOutputCP(CP_UTF8);
18     #endif
19     // lab1 --seed N повторяет запуск с теми же случайными числами
20     unsigned seed = make_seed();
21     if(argc == 3 && std::strcmp(argv[1], "--seed") == 0){
22         seed = static_cast<unsigned>(std::strtoul(argv[2], nullptr, 10));
23     }
24     seed_random(seed);
25     std::cout << "seed = " << seed << std::endl;
26
27     try{
28         Menu menu;
29         menu.run();
30     }
31     catch(const InputClosedError&){
32         std::cout << std::endl << "Ввод закончился" << std::endl;
33     }
34     catch(const std::exception& error){
35         std::cout << "Внутренняя ошибка: " << error.what() << std::endl;
36         return 1;
37     }

```

```

36     }
37     return 0;
38 }

```

Листинг 17 — tools/bench.cpp

```

1  // Замер времени самых долгих действий программы при разных разрядностях n.
2  // По нему выбрана MAX_DEPTH: наибольшее n, при котором любое действие
   занимает меньше 1 с.
3  // Сборка: g++ -std=c++14 -O2 -Isrc tools/bench.cpp src/core/*.cpp -o bench
4  #include "core/GrayCode.hpp"
5  #include "core/Multiset.hpp"
6  #include "core/Random.hpp"
7  #include "core/Universe.hpp"
8  #include <algorithm>
9  #include <chrono>
10 #include <iomanip>
11 #include <iostream>
12 #include <memory>
13
14
15 // наименьшее из трёх измерений, мс: так меньше сказываются посторонние
   процессы
16 template<class Action>
17 static double measure(Action action){
18     double best = 1e18;
19     for(int run = 0; run < 3; run++){
20         auto start = std::chrono::steady_clock::now();
21         action();
22         auto finish = std::chrono::steady_clock::now();
23         best = std::min(best, std::chrono::duration<double,
           std::milli>(finish - start).count());
24     }
25     return best;
26 }
27
28 int main(){
29     seed_random(1);
30     std::cout << " n   код Грея   универсум   заполнение A   13 операций
   наибольшее, мс" << std::endl;
31     for(size_t depth = 10; depth <= MAX_DEPTH + 2; depth++){
32         std::shared_ptr<const CodeSet> carrier;
33         double t_code = measure([&]{ carrier =
           std::make_shared<CodeSet>(depth); });
34
35         // наихудший случай: все кратности равны MAX_MULTIPLICITY
36         Multiset universe(carrier);

```

```

37     double t_universe = measure([&]{ universe = uniform_universe(carrier,
38                                     MAX_MULTIPLICITY); });
39
40     // заполнение делает |A| шагов, поэтому дольше всего при |A| = |U|
41     Multiset a(carrier);
42     Multiset b(carrier);
43     double t_fill = measure([&]{ a = random_submultiset(universe,
44                                     universe.cardinality()); });
45     a = random_submultiset(universe, universe.cardinality() / 2);
46     b = random_submultiset(universe, universe.cardinality() / 3);
47
48     size_t checksum = 0;
49     double t_operations = measure([&]{
50         checksum += a.unite(b).cardinality() +
51                     a.intersect(b).cardinality()
52                     + a.difference(b, universe).cardinality() +
53                     b.difference(a, universe).cardinality()
54                     + a.symmetric_difference(b, universe).cardinality()
55                     + a.complement(universe).cardinality() +
56                     b.complement(universe).cardinality()
57                     + a.arithmetic_sum(b, universe).cardinality() +
58                     a.arithmetic_difference(b).cardinality()
59                     + b.arithmetic_difference(a).cardinality() +
60                     a.arithmetic_product(b, universe).cardinality()
61                     + a.arithmetic_division(b).cardinality() +
62                     b.arithmetic_division(a).cardinality();
63     });
64
65     double worst = std::max({t_code, t_universe, t_fill, t_operations});
66     std::cout << std::fixed << std::setprecision(1) << std::setw(2) <<
67     depth
68         << std::setw(11) << t_code << std::setw(12) << t_universe
69         << std::setw(15) << t_fill
70         << std::setw(14) << t_operations << std::setw(17) << worst
71         << (worst < 1000 ? "" : " > 1 c") << std::endl;
72     if(checksum == 0) std::cout << "";    // не даёт компилятору выбросить
73     операции
74 }
75 return 0;
76 }

```